



**AeroInsight BI – Flight Operations & Analytics Using Data
Analysis : A Case Study on US Airways Inc.**

Submitted in partial fulfilment of the requirements for the award of
the degree of

Master of Science (Data Science)

By

Vanshika Aggarwal

UID : O24MSD110207

Under the guidance of

Mr. Kunwar Saurabh Bisen

BONAFIDE CERTIFICATE

This is to certify that the project entitled

**“AeroInsight BI -Flight Operations & Analytics Using Data Analysis: A
Case Study on US Airways Inc.”**

is a bonafide work carried out by

VANSHIKA AGGARWAL

UID : O24MSD110207

In partial fulfilment of the requirements for the award of the degree of Master Of Science in Data Science from Chandigarh University, Mohali, Punjab. This project has been carried out under supervision and has not been submitted to any other university or institution for the award of any other degree or diploma.

Guide Name: Mr. Kunwar Saurabh Bisen

DECLARATION

I, Vanshika Aggarwal, hereby solemnly declare that the project report titled “AeroInsight BI – Flight Operations & Analytics using Data Analysis: A Case Study on Us airways Inc.” submitted in partial fulfilment of the requirements for the award of the degree of Master of Science (Data Science) is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2024-26 under the supervision of Mr. Kunwar Saurabh Bisen.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected, and disciplinary action may be taken as per institutional rules.

Place: Delhi

Date: 4th April 2026



Vanshika Aggarwal

UID : O24MSD110207

ACKNOWLEDGEMENT

I would like to thank my project guide Mr. Kunwar Saurabh Bisen for his consistent guidance, helpful suggestions, and directions throughout the duration of this project. The kind advice at every step, from collecting the data till deploying the dashboard has been very useful in creating this project.

I would also like to acknowledge the Bureau of Transportation Statistics (BTS), United States Department of Transportation, for maintaining and making available online the aviation data, namely T-100 Domestic Segment, Form 41 Financial Schedules, and On-Time Performance data which was the basis of this study. Without open access to such rich, real-world data, a project of this scope and depth would simply not have been possible.

And, lastly, I am forever indebted to my family and friends who gave me immense support in overcoming all the difficult phases of the project.

Vanshika Aggarwal
M.Sc. Data Science
Chandigarh University

ABSTRACT

Tremendous volumes of operational and financial data are generated within the airline industry daily; however, most organizations still fail to effectively transform this data into business intelligence. AeroInsight BI - Flight Operations & Analytics Using Data Analysis: A Case Study on US Airways Inc. Attempts to bridge this gap through the creation of a fully interactive BI dashboard, powered by R Shiny and utilizing live aviation data from the Bureau of Transportation Statistics (BTS).

The case study used the fifth largest commercial airline operating in the United States at the time, US Airways Inc. And covered a critical four-year time span, from 2010 to 2013 - just prior to the airline's merger with American Airlines in December 2013. Four separate datasets from the BTS were analysed for this case study: the T-100 Domestic Segment dataset (42,142 records, covering routes), Form 41 Schedule P-5.2 dataset (aircraft operating expenses), Form 41 Schedule P-12A dataset (fuel cost and usage), and the Reporting Carrier On-Time Performance dataset (1,632,669 individual flight records).

The main financial KPIs analysed and calculated were CASM, Load Factor, the proportion of total operating expenses due to fuel costs, and a new Delay Cost Estimation, calculated at the widely used rate of \$100/min. The interactive dashboard features 9 analytical tabs where financial managers and aviation analysts may use the BI tools to evaluate route networks, identify inconsistencies within hub operations, analyze financial alternatives through the interactive What-If Simulator, and assess the historical performance against the present-day industry standard.

Key financial outcomes include; average CASM of 10.65 cents (lower than the 2024 industry average of 11.2 cents, as reported in the most recent aviation economic datasets (*Airlines for America (A4A, 2023)*)), overall load factor of 81.2% (higher than the industry average of 80%), total system wide operating expenses, of which fuel was 17.9% (lower than the industry average of 30% likely implying effective fuel hedging), and estimated costs due to delays totalling approximately \$1.44B from 2010-2013, 22.3% of which were deemed controllable by the carrier.

This study concludes that strategically significant financial insights, potentially on par with what would be provided by a niche aviation consultancy, can be generated using readily available public government data, if modelled and presented through a well-designed analytical framework and interactive BI dashboard.

Keywords: Business Intelligence, Aviation Analytics, R Shiny, CASM, Load Factor, US Airways, BTS, Data Validation, Financial Decision-Making, Delay Cost Analysis

Table of Contents

BONAFIDE CERTIFICATE.....	i
DECLARATION.....	ii
ACKNOWLEDGEMENT.....	iii
ABSTRACT.....	iv
LIST OF TABLES	ix
LIST OF FIGURES.....	x
LIST OF ABBREVIATIONS.....	xi
Chapter 1 : Introduction	1
1.1 Background of the Project	1
1.2 Problem Statement	2
1.3 System Objectives	3
1.4 Project Scope	3
1.5 Review of Existing System	4
1.6 Proposed System.....	6
1.7 Technologies Employed.....	6
Chapter 2 : System Study.....	8
2.1 Aviation Analytics and Financial Performance Measurement.....	8
2.2 Business Intelligence Systems in Transportation	9
2.3 R Shiny as Business Intelligence Platform	10
2.4 Related Works and Similar Projects	10
2.5 Research Gap and Contribution of This Project.....	11
Chapter 3 : System Analysis	13
3.1 Functional Requirements.....	13
3.1.1 Data Management Requirements.....	13
3.1.2 Analytical Requirements	13
3.1.3 Dashboard Interface Requirements.....	14
3.2 Non-Functional Requirements	15
3.3 Feasibility Study	15
3.3.1 Technical Feasibility.....	15
3.3.2 Economic Feasibility.....	16

3.3.3	Operational Feasibility	16
3.4	Data Flow Diagram – Level 0 (Context Diagram)	17
3.5	Data Flow Diagram – Level 1.....	18
3.6	Use Case Diagram	18
Chapter 4	: System Design	20
4.1	System Architecture	20
4.2	Data Pipeline Architecture	21
4.3	Database Schema Design	22
4.4	UML Class Diagram	24
4.5	Sequence Diagram	24
4.6	Activity Diagram	25
4.7	Entity Relationship Diagram.....	26
4.8	KPI Definitions and Formulae.....	27
Chapter 5	: System Implementation	29
5.1	Development Environment.....	29
5.2	Hardware and Software Requirements.....	30
5.2.1	Hardware Requirements	30
5.2.2	Software Requirements	30
5.3	Project Directory Structure	30
5.4	Module-Wise Implementation.....	31
5.4.1	Module 1: Data Preparation (data_prep.R)	31
5.4.2	KPI Calculation Engine (kpi_calc.R)	32
5.4.3	Module 3: Chart Library (plots.R)	33
5.4.4	Module 4: Financial What-If Simulator	35
5.5	Dashboard Screenshots	36
5.6	CSS Styling and UI Design.....	38
Chapter 6	: Testing	40
6.1	Testing Strategy	40
6.2	Unit Testing – Data Loading Module	40
6.3	Unit Testing -KPI Calculation Module.....	41
6.4	Integration Testing	42
6.5	System Testing.....	43

6.6	Bugs Encountered and Resolved	44
Chapter 7	: Results & Discussion	46
7.1	KPI Summary	46
7.2	CASM Analysis and Discussion	46
7.3	Load Factor Analysis.....	47
7.4	Hub Performance Results.....	48
7.5	Delay Cost Analysis.....	49
7.6	Financial Simulator Results	49
7.7	Comparison with Existing Results	50
Chapter 8	: Conclusion and Future Enhancement.....	52
8.1	Project Summary.....	52
8.2	Objectives Achievement Review	52
8.3	Key Technical Learning Outcomes.....	52
8.4	Conclusions	53
8.5	Future Enhancements.....	54
Chapter 9	: References.....	56
Chapter 10	: Appendices	58
Appendix A:	Complete Source Code	58
A.1	data_prep.R – Data Preparation Pipeline	58
Appendix B:	Data Schema	61
Appendix C:	Installation Guide.....	62
Appendix D:	User Manual	62

LIST OF TABLES

Table 1.1 Comparison of Existing Airline BI Tools with AeroInsight BI.....	5
Table 2.1 Summary of Literature Review	11
Table 3.1 Non-Functional Requirements of AeroInsight BI.....	15
Table 4.1 T-100 Domestic Segment - Key Fields After Processing.....	23
Table 4.2 KPI Definitions & Formulae.....	27
Table 5.1 Development Environment Specifications.....	29
Table 5.2 Hardware Requirements.....	30
Table 5.3 R Package Dependencies	30
Table 5.4 Directory Structure.....	31
Table 6.1 Unit Test Case - Data Loading Module.....	40
Table 6.2 Unit Test Cases - KPI Calculation Module	41
Table 6.3 Integration Test Cases	42
Table 6.4 System Test Cases - End-to-End Dashboard Testing	43
Table 7.1 Annual KPI Summary - US Airways 2010-2013	46
Table 7.2 Hub Performance Comparison - Key Metrics (2013).....	48
Table 7.3 Financial Simulator - Pre-Defined Scenario Results	49
Table 8.1 Project Objectives	52

LIST OF FIGURES

Figure 3.1 Level 0 DFD - AeroInsight BI Context Diagram	17
Figure 3.2 Level 1 DFD - AeroInsight BI Internal Data Flow.....	18
Figure 3.3 Use Case Diagram - AeroInsight BI System	19
Figure 4.1 System Architecture Diagram.....	20
Figure 4.2 Data Pipeline Architecture.....	22
Figure 4.3 Class Diagram - Module Structure	24
Figure 4.4 Sequence Diagram.....	25
Figure 4.5 Activity Diagram	26
Figure 4.6 ER Diagram - AeroInsight BI Conceptual Data Model.....	27
Figure 5.1 Executive Overview Tab - Four KPI cards and CASM quarterly trend.....	37
Figure 5.2 Route Analytics Tab - Color-Coded Route Map.....	37
Figure 5.3 Cost Simulator Tab - showing \$2.58B Estimated Savings.....	38
Figure 7.1 CASM Trend - US Airways 2010-13 (Quarter)	47
Figure 7.2 Route Analytics Tab - Load Factor Distribution	48

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ASM	Available Seat Miles
BI	Business Intelligence
BTS	Bureau of Transportation Statistics
CASM	Cost per Available Seat Mile
CLT	Charlotte Douglas International Airport
CSV	Comma Separated Values
DFD	Data Flow Diagram
ER	Entity Relationship
FAA	Federal Aviation Administration
GUI	Graphical User Interface
KPI	Key Performance Indicator
LF	Load Factor
MSc	Master of Science
NAS	National Airspace System
PHL	Philadelphia International Airport
PHX	Phoenix Sky Harbor International Airport
R	R Programming Language
RASM	Revenue per Available Seat Mile
RDS	R Data Serialization (file format)
SQL	Structured Query Language
UI	User Interface
UML	Unified Modelling Language
URL	Uniform Resource Locator
US	US Airways (Carrier Code)
USDOT	United States Department of Transportation
YoY	Year-over-Year

Chapter 1 : Introduction

1.1 Background of the Project

The global airline industry holds an unusual place in the world economy – it is both one of the most data-intensive sectors and one of the most financially unstable (*International Air Transport Association (IATA)*, 2023). Every day, airlines produce millions of records detailing nearly every facet of their operations: flight departures, fuel usage, passenger capacity, maintenance spending, delay durations and crew allocations. Despite generating this vast sea of information, many carriers, - especially mid-sized ones with limited analytical capacity - have long struggled to turn this raw data into actionable financial insight.

During the early 2010s, U.S. carriers had a lot of trouble. The industry was still trying to bounce back from the drop in demand that followed the 2008 financial crisis. US Airways case study deals with a situation where oil prices are witnessing a big change and the passengers are demanding faster electronic communication and transparency. In this scenario, being able to make faster, data-driven financial decisions were not just a competitive advantage, but a matter of life and death.

US Airways was one of the biggest airlines in the US from 2010 to 2013 and it was one of the airlines that flew through this area. In 1939 the company started its operation by naming the Allegheny Airlines. Subsequently, in 1996 the airline's branding underwent a final change, and the airline was named US Airways. It flew more than 3,000 times a day to over 200 U.S. destinations. The airline's main hubs were Philadelphia (PHL), Charlotte (CLT), and Phoenix (PHX). During this time, it was the fifth busiest airline in the country for passengers.

This project focuses on US Airways for several key analytical reasons. First, the 2010–2013 timeline offers a self-contained operational cycle that begins with post-recession stabilization and ends with the airline's merger announcement with American Airlines, finalized in December 2013. This makes the data set ideal for a longitudinal financial study. Second, the publicly available Bureau of Transportation Statistics (BTS) data for this carrier is highly detailed and well-structured for multi-dimensional financial analysis — exactly the type of data the AeroInsight BI system is designed to handle. Finally, the case is economically interesting: subtle operational signals, such as a falling Cost per Available Seat Mile (CASM) in 2013, mirror the financial logic that would later underpin the merger.

Business Intelligence (BI) refers to the process of converting operational data into structured, decision-supporting information. In an airline setting, BI helps answer crucial financial questions: Which routes are least profitable relative to costs? How effectively are hub airports

utilized? What share of costs stems from factors management can control? How do swings in fuel prices cascade through the cost base? Such insights must be available constantly and interactively, not just during annual strategic reviews.

AeroInsight BI was created to meet that demand. Built on the R Shiny framework, it combines data from four major BTS datasets — operational traffic, aircraft expenses, fuel consumption, and on-time performance — into one interactive analytics environment containing nine modules. The system calculates and visualizes five major financial key performance indicators (KPIs), supports dynamic filtering and drill-down, includes a custom financial “What-If” simulator, and benchmarks historical figures against current industry averages.

1.2 Problem Statement

Although the Bureau of Transportation Statistics maintains extensive open aviation data, there remains a shortage of integrated, user-friendly analytical tools for investigating the financial efficiency and operational performance of individual carriers. BTS information is fragmented across multiple formats — traffic, cost, fuel, and delay datasets — each with its own time intervals and carrier codes. Combining them into one coherent framework demands technical expertise that smaller teams and academic research groups often lack.

Existing studies and tools generally suffer from the same limitation: they either take a high-level industry view or focus on narrow operational metrics. What’s missing is a holistic BI solution capable of simultaneously analyzing cost efficiency (CASM), capacity utilization, fuel cost dynamics, hub-level differences, and the financial burden of delays, in one interactive dashboard that does not require coding knowledge.

This project closes that gap by designing and developing AeroInsight BI, a specialized R Shiny system that unifies four major BTS data sources for US Airways (2010–2013), computes five critical KPIs using standardized formulas, and presents them through a nine-tab dashboard featuring filters, drill-downs, scenario testing, and benchmarking.

The challenge is not merely technical. Even when data is available and tools exist, the translation of raw aviation records into financial insight requires domain-specific KPI frameworks, careful multi-dataset integration, and presentation mechanisms that allow non-technical financial managers to interact with complex analytical outputs. Standard spreadsheet tools and generic reporting platforms fail to meet this need because they lack the combination of statistical computing power, interactive visualisation, and custom KPI flexibility that aviation financial analysis demands.

Moreover, the 2010-2013 period for US Airways presents a particularly instructive case precisely because the airline was navigating simultaneous pressures – post-recession demand recovery, fuel price volatility, and pre-merger operational pressures restructuring – that each left distinct and measurable fingerprints in the financial data. Analysing this period through a structured BI framework allows those fingerprints to be identified, quantified and interpreted in ways that static reporting could never achieve.

1.3 System Objectives

The AeroInsight BI was built to serve four main objectives in synchrony with the academic objectives: -

- Show the application of Business Intelligence tools (R Shiny) to analyze aviation financial data and show the use of open-source business intelligence tools to deliver enterprise-grade analytics leveraging open, public data sources.
- To apply data cleansing and integration, aggregation and condensation, KPI computation and statistical data summarization techniques to extract valuable financial patterns and operational trends from the 2010-2013 US Airways flight data.
- To develop an interactive Financial What-If Simulator tool that proves how data analytics can help the aviation industry make more informed financial decisions by assessing the financial impact of various changes to core operations such as fleet efficiency changes, fuel prices and delays.
- To develop a thorough, academic report and a working, deployable Business Intelligence dashboard to document the entire project lifecycle from data acquisition and preprocessing to system design, implementation, testing, and interpretation of results.

1.4 Project Scope

The project scope is defined along four dimensions namely – temporal, geographic, analytical, and technical.

Temporally, the analysis is confined to the four calendar years from January 2010 through December 2013 which were judged to be US Airways' most recent and stable years of operations prior to its merger and consolidation with American Airlines. Data from 2014 onwards when US Airways and American Airlines began their joint operations under the AA

carrier code post-merger are not included. Doing so would create confounding effects analytically inconsistent with a case study of US Airways alone.

On a regional basis, the T-100 traffic analysis includes only US Airways domestic system operations (within the conterminous United States and its territories), because international operations are regulated, managed for yield and fares by different systems. The P-5.2 and P-12A financial datasets have been included for domestic and international systems, as well as totals were applicable.

Analytical inputs include using five financial KPIs: CASM, Load Factor, Fuel Cost as a percentage of total operating expense, average Fuel Price (per gallon), estimated Delay Cost using the \$100 per delay industry standards . Revenue-side metrics [Revenue per Available Seat Mile (RASM), route-level profitability] are excluded since the BTS only publicly releases them on an aggregate level, rather than at a route-level.

Technically, the dashboard is built entirely in R using the interactive web framework R Shiny, the tidyverse and data.table R packages, Plotly, Leaflet and the DT R package for interactive data tables. It is designed to run on shinyapps.io. The application does not use a database backend and loads data into memory from pre-processed RDS files at application startup.

It is also important to note what fails outside the scope of this project. The analysis is restricted to US Airways' domestic operations as reported to the BTS; international route data and code-share operations are not included. The study period ends at December 2013, prior to the legal completion of the merger with American Airlines, and therefore does not address post-merger integration outcomes. Revenue-side metrics such as Revenue per Available Seat Mile (RASM) and yield management performance are not computed, as the BTS DB1B origin-destination ticket database was not incorporated into this version of the system. These exclusions are deliberate and are noted here so that the scope boundaries are transparent to any reader seeking to extend or replicate this analysis.

1.5 Review of Existing System

Before development began, the project reviewed current analytical tools in the airline industry to pinpoint shortcomings. These could be grouped into three broad categories:

- **Commercial BI tools (Tableau, Power BI, IBM Cognos):** Strong but expensive proprietary systems used by major airlines. They require expert configuration for aviation-specific analysis and are largely inaccessible to smaller teams or researchers.

- **Specialized aviation solutions (Sabre AirVision, PROS Revenue Management, Amadeus Revenue Integrity):** Powerful but costly closed-source suites aimed at large carriers' revenue management departments, offering little utility for academic study.
- **Academic or open-source efforts:** Many researchers use BTS data, but most studies result in static reports or basic charts rather than fully interactive, deployable BI dashboards. No open-source alternative exists that integrates all four key BTS datasets into a unified financial analysis platform.

Table 1.1 presents a structured comparison of existing airline BI tools and academic approaches against the capabilities of AeroInsight BI.

Table 1.1 Comparison of Existing Airline BI Tools with AeroInsight BI

Feature	Tableau	Power BI	Academic Studies	AeroInsight BI
BTS Data Integration	Partial	Partial	Partial	Full
Interactive Route Map	Yes	Yes	No	Yes
CASM Calculation	Manual	Manual	Sometimes	Automated
Delay Cost API	No	No	Rare	Yes (Original)
Financial Simulator	No	No	No	Yes (Original)
Open Source	No	No	N/A	Yes
Cost	High	Medium	Free	Free
Aviation-Specific Design	No	No	Partial	Yes

What this comparative review makes clear is that the existing landscape of aviation analytics tools presents a binary choice: either powerful but expensive proprietary platforms accessible only to large carriers, or flexible open-source tools that require substantial manual effort to configure for aviation-specific use. AeroInsight BI is designed to occupy the space between these two extremes – providing aviation-specific KPI computation and interactive financial simulation within a fully open, reproducible, and cost-free framework. This positioning is what makes it a genuine contribution rather than merely a replication of existing approaches.

1.6 Proposed System

AeroInsight BI is designed as a holistic, interactive, open-source BI dashboard that resolves the deficiencies identified above. It integrates four BTS datasets into a single analytics environment, automatically computes five KPIs, and organizes the results under nine dedicated dashboard tabs.

The architecture follows a three-tier structure:

- **Data layer:** Houses the four pre-processed BTS datasets in compressed RDS format, initialized at startup via global.R.
- **Analytical layer:** Implements KPI calculations, dataset merging, and plotting logic via scripts such as `kpi_calc.R` and `plots.R` using reusable function libraries.
- **Presentation layer:** Defined in `app.R`, this layer constructs the Shiny interface, linking user inputs to real-time visual responses.

More involved is a key component of the proposed system, the Financial What-If Simulator, which enables users to investigate the impacts of three simultaneous operating scenarios: CASM reduction through fleet efficiency, fuel price scenario changes and delay reduction through operational efficiencies. The Financial What-If Simulator seeks to turn the dashboard from a passive data exploration tool to an active financial decision-support platform.

1.7 Technologies Employed

The following technologies are used in AeroInsight interface: -

- **R Programming Language (version 4.5):** The main programming language that is used for all data processing and statistical computing as well as application development. R has got an extensive ecosystem of packages which makes it very suitable for data science and BI.
- **R Shiny:** The open-source web application framework for R is used to build interactive web dashboards. It doesn't require knowledge of HTML, CSS and java script at a fundamental level. Shiny utilizes a reactive programming pattern so that dashboard elements automatically update after user inputs.
- **tidyverse / data.table :** The tidyverse consists of R packages (`dplyr`, `ggplot2`, `tidyr`, `lubridate` etc.) for data manipulation and visualisation, following the 'tidy data' principles established for modern data science (Grolemund & Wickham, 2017). `data.table` is also an R package for the purpose. `data.table` facilitates fast data loading

and aggregation, which is crucial for the 1.6 million row On-Time Performance data set.

- **Plotly for R:** It is an interactive charting library that allows creating eye-catching, hover-enabled, zoom-capable charts directly from R. Chart rendering on all nine dashboard tabs is done through Plotly.
- **Leaflet:** It is an easy way to create interactive web maps using open-source JavaScript library leaflet. For the Route Analytics tab a clean cartographic aesthetic based on CartoDB Positron tiles forms the basis of the maps on the interactive route map.
- **DT (Data Tables):** It is the jQuery DataTables library R wrapper having an excellent R interface which easily lets you display tables that are searchable, sortable and interactive for financial data tables.
- **BTS Transtats:** The BTS Transtats (Bureau of Transportation Statistics) which provides four aviation data sets that is available for public use is the principal data source for the project. US certification is used by all these accolades.

Chapter 2 : System Study

The need for an extensive review of the current literature cannot be understated when considering any research endeavour such as this one. This chapter presents a examination of what has been published, built, and documented within the intersecting fields of aviation analytics, business intelligence platforms, key performance indicators in financial reporting for the transport industry, and the language R used in data science applications. This chapter has been structured based on five themes, each of which has been influenced a particular design choice of AeroInsight BI.

2.1 Aviation Analytics and Financial Performance Measurement

There have been many works published on financial performance in the airline industry, but many of these predate the existence of the detailed operational data that BTS now publishes to the public. In terms of methodology, early works used to employ mostly aggregate annual information, whereas today's datasets consist of information for each single flight.

An especially influential text on airline operations management was written by (Belobaba et al., 2009): The Global Airline Industry. This textbook gives an extremely comprehensive overview of airline operations management. In particular, the approach of decomposing airline costs into the subcategories of flights operations costs, maintenance costs, and costs of aircraft ownership, among others, inspired the design of the KPI framework of the AeroInsight BI system. Indeed, the cost categories used in the BTS Form 41 Schedule P-5.2 data series correspond nearly perfectly to those proposed by the textbook authors.

In his book *Straight and Level: Practical Airline Economics*, (Holloway, 2008) asserts that CASM is the most critical airline management performance measure because it standardizes the cost measures against the size and network of the carrier involved. The justification of this theory helped in deciding the choice of using CASM as the focus of analysis in AeroInsight BI. Another concept highlighted in this book by Holloway regarding load factor is that this financial performance indicator should be viewed in relation to revenues as well, where its correlation to yield is non-linear and route dependent.

The discussion by (Vasigh et al., 2013) on financial analysis of airlines from the view point of investment and managerial accounting also holds a lot of significance in light of the results of this study, which revealed that US Airways managed to keep their fuel expense ratio at 17.9%

in relation to their overall operating costs. This figure is much lower than the 30% average rate seen in unhedged carriers in the industry.

In more recent studies on this topic, (Rupp & Holmes, 2006) and (Mayer & Sinai, 2003) analyze the economic consequences of delays at the airlines and establish empirically based relationships between delay frequency and duration and the economic performance of airlines. This research constitutes the scholarly basis for the method of delay cost evaluation used in this study, which is built upon the FAA standard of charging \$100 per delay minute established by Airlines for America.

2.2 Business Intelligence Systems in Transportation

Use of BI tools in the domain of transportation and logistics has attracted considerable scholarly interest from the early 2000s; however, use of BI tools in aviation has been less researched compared to road freight and rail transport.

In the book titled Performance Dashboards: Measuring, Monitoring, and Managing Your Business, (Eckerson, 2010) provides the theoretical base for operational dashboards, which guided the development of the AeroInsight BI dashboard. In the book, (Eckerson, 2010) categorises the use of dashboards into three different types. First is the strategic dashboard, second is the tactical dashboard, and third is the operational dashboard. Strategic dashboards help measure long-term organisational goals, tactical dashboards measure weekly or monthly progress of the departments, and operational dashboards measure real time or near-real time operations.

(Few, 2006) in "Information Dashboard Design" stresses the significance of cognitive efficiency in dashboard design, which states that the insights from a dashboard should be immediately understandable upon looking at the dashboard without requiring any mental arithmetic or additional information. Some of the design aspects influenced by this aspect include placing KPI value boxes prominently in the Executive Overview tab, colour-coded load factors in the route map, and contextual interpretation text in the Live Benchmarks tab.

In the literature related to BI in aviation, (Rajasekar & Thampuran, 2014) provide examples of using data warehousing methods to consolidate and analyze airline operations data. The data warehousing example used by the authors in their paper is equivalent to the data preparation pipeline designed in the `data_prep.R` script in the current project. However, unlike the data warehousing example, AeroInsight BI uses a simpler method with the help of `data.table` and `tidyverse` libraries in R.

2.3 R Shiny as Business Intelligence Platform

Whereas R has been utilized as an environment for statistical computing since the 1990s, the use of R as an interactive BI dashboard platform has only become common in recent years due to the emergence of the Shiny framework, which was developed by RStudio (now known as Posit) in 2012.

Mastering Shiny is the definitive work on Shiny frameworks by (Wickham, 2021). This text covers aspects such as the reactive programming paradigm, module structures, and optimization processes that have contributed to the building of AeroInsight BI. The separation between `global.R`, `server.R`, and `ui.R` files, using `reactive()` for filters to update the charts, and `eventReactive()` for the financial simulator are based on the insights provided by Wickham.

(Chang et al., 2024) and the Shiny development team have pointed out that in data-intense applications like dashboards, the biggest performance overhead is not reactive computations but rather the loading of data at startup. It was for this reason that in AeroInsight BI the approach taken was to compute the KPIs in advance within `kpi_calc.R` and to save them into RDS files, which can then be loaded using `global.R` upon application start-up.

In his book *Interactive Web-Based Data Visualization with R, plotly, and shiny*, (Sievert, 2020) gives detailed instructions for integrating Plotly and Shiny together, instructions which guided the development of the charts used within AeroInsight BI. In particular, Sievert's insights on hover templates, colour scales, and layout theme were especially helpful.

2.4 Related Works and Similar Projects

Several associated studies can be cited for comparative analysis of the proposed tool, as follows:

(Sun et al., 2017) conducted an analysis of BTS data concerning on-time performance of flights, creating several static visualizations with the help of Python and Pandas. It is an excellent piece of research work; however, it creates static images only and does not consider financial costs. Therefore, the current research is different from this one by offering a more advanced tool with the ability to create dynamic visualizations.

(Agnihotri & Bhatt, 2019) created an airline dashboard with the use of Tableau, which considers information provided by the BTS and International Air Transport Association

(IATA). It was a dashboard aimed at analyzing such metrics as market share and routes competition between airlines. This study provides evidence on the possibility of creating a successful dashboard for analyzing aviation data using Tableau; however, the latter's licensing prevents the use of their tool further.

In the realm of open source development, Apporv Thite (2026) built AeroInsight, a Next.js and FastAPI powered system relying on BTS data to compare carriers across the entire industry. The advantage of the project was the usage of machine learning technology – a recommendation engine for aircraft types based on XGBoost with an impressive 93.7% accuracy rate. However, AeroInsight BI project has a completely different angle: it concentrates on the financial analysis of one particular carrier, offering features like CASM breakdowns, delay costs assessment, and financial scenarios simulation.

2.5 Research Gap and Contribution of This Project

The review of existing literature and relevant studies indicates that there exists a clear gap: No existing free, public platform combines all four datasets of BTS aviation in one single carrier-specific financial BI dashboard that not only incorporates CASM analysis, load factor tracking, fuel cost tracking, and delay cost calculation but also includes interactive financial modeling tools. This is exactly the gap that AeroInsight BI intends to fill.

This project brings original contributions in three distinct areas. Firstly, the creation of an integrated four-datasets data pipeline for US Airways which deals with discrepancies regarding unit inconsistencies, temporal granularity, and carrier codes that have previously posed technical difficulties when combining multiple datasets from BTS aviation. Secondly, the creation of a new financial delay cost KPI that utilizes the delay cost estimation method used by airlines and converts it into financial terms on a monthly basis to facilitate trend analysis. Thirdly, the creation of an interactive Financial What-If Simulator that enables users to run financial scenarios by manipulating three parameters at once.

Table 2.1 Summary of Literature Review

Author(s) & Year	Focus Area	Method/Tool	Relevance to This Project
(Belobaba et al., 2009)	Airline Cost Structure	Textbook Analysis	CASM framework and cost taxonomy

(Holloway, 2008)	Airline Economics	Financial Analysis	CASM as primary KPI justification
(Vasigh et al., 2013)	Financial Management	Accounting Methods	Fuel hedging interpretation
(Rupp & Holmes, 2006)	Delay Impact	Empirical Study	Delay cost methodology basis
(Eckerson, 2010)	BI Dashboard Design	Design Framework	Dashboard architecture principles
(Few, 2006)	Dashboard UX	Design Theory	Cognitive efficiency in UI design
(Wickham, 2021)	R Shiny Framework	R Programming	Shiny architecture decisions
(Sievert, 2020)	Plotly with Shiny	R + JavaScript	Chart implementation guidance
(Sun et al., 2017)	OTP Analysis	Python/Pandas	Predecessor; no financial integration

Chapter 3 : System Analysis

The process whereby the requirements, scope, and feasibility of a particular system to be implemented are defined is referred to as system analysis. In this chapter, we describe the requirements analysis, feasibility study, and modelling of the proposed system - AeroInsight BI. System analysis involves an organized way of determining not only the functional requirements of a system but also its non-functional requirements using various means of feasibility studies and demonstrating them using data flow diagrams and use case diagrams.

3.1 Functional Requirements

The functionalities required by the system can be described by what the system should do for the benefit of its users. The list of functional requirements below was developed based on an analysis of stakeholders (academics who supervise the project, and domain experts from aviation finance) and comparative systems analysis.

3.1.1 Data Management Requirements

- **FR-DM-01:** The system will load and process data from four BTS databases: T-100 Domestic Segment database, Form 41 Schedule P-5.2, Form 41 Schedule P-12A, and Reporting Carrier On-Time Performance database.
- **FR-DM-02:** The system will filter all databases to keep only those data that correspond to US Airways (carrier 'US') between January 2010 and December 2013.
- **FR-DM-03:** The system will manage any type conflicts found in the On-Time Performance database (due to monthly data concatenation) by converting all columns into characters and later explicitly converting numerical columns.
- **FR-DM-04:** All KPI computations should be performed prior to their use and saved as compressed RDS files to load in less than one second on the dashboard.

3.1.2 Analytical Requirements

- **FR-AN-01:** The system must calculate CASM (Total Operating Cost/Total ASM) on a quarterly basis and break down into fuel, labor, and maintenance components.
- **FR-AN-02:** The system must calculate Load Factor (Passengers/Seats) on a route-monthly basis and aggregate at the monthly, quarterly, and yearly levels.

- **FR-AN-03:** The system must calculate the Fuel Cost Percentage (Fuel Cost/Total Operating Cost) on a quarterly basis.
- **FR-AN-04:** The system must calculate the Delay Cost on a monthly basis using the equation Total Delayed Time * \$100/min.
- **FR-AN-05:** The system must calculate the performance indicators of each of the PHL, CLT, and PHX hubs separately.

3.1.3 Dashboard Interface Requirements

- **FR-UI-01:** The system should have an Executive Overview tab containing four boxes for KPIs, CASM trend graph, cost pie chart, monthly trend graph of load factor, and key findings.
- **FR-UI-02:** The system should have a Route Analytics tab containing year and hub filter buttons, Leaflet map of route, top 20 routes bar graph, and load factor distribution graph.
- **FR-UI-03:** The system should have a Fleet Cost Analysis tab containing stacked graph of CASM elements, CASM year-over-year trend graph, and quarterly expenses table (searchable).
- **FR-UI-04:** The system should have a Fuel Intelligence tab containing fuel price trend graph, graph of fuel cost compared to total costs, and fuel cost trend with industry benchmark.
- **FR-UI-05:** The system should have a Hub Performance tab comparing PHL, CLT, and PHX for passengers, load factor, routes, departure, and performance scorecard.
- **FR-UI-06:** The system should have a Delay Cost Impact section displaying trends in delay costs, reasons for delays, and comparison between delays caused by carriers and weather.
- **FR-UI-07:** The system should have a Financial What-If simulator with three interactive sliders and a results area.
- **FR-UI-08:** The system should have a Live Benchmarks section showing the comparison between US Airways and industry average for 2024.
- **FR-UI-09:** The system should have an About This Project section that will describe the objectives, dataset used, and key findings.

3.2 Non-Functional Requirements

Table 3.1 consolidates all the non-functional requirements of the interface: -

Table 3.1 Non-Functional Requirements of AeroInsight BI

ID	Category	Requirement	Metric
NFR-01	Performance	Dashboard shall load fully within 10 seconds on standard hardware	<10 seconds
NFR-02	Performance	Chart updates following filter changes	<2 seconds
NFR-03	Reliability	System shall handle missing values without crashing using na.rm=TRUE	Zero crashes
NFR-04	Usability	All charts shall include hover tooltips with contextual data	100% coverage
NFR-05	Usability	Color schemes shall be consistent with US Airways brand colors	Navy / Red
NFR-06	Compatibility	Dashboard shall render correctly in browsers such as Chrome and Edge.	3 browsers
NFR-07	Maintainability	All r codes shall include inline comments explaining logic	100% functions
NFR-08	Scalability	Data pipeline shall support addition of new year's without restructuring	Modular design
NFR-09	Portability	Dashboard shall be deployable on shinyapps.io without modification	Zero config

3.3 Feasibility Study

3.3.1 Technical Feasibility

Feasibility of the project from the technical point of view can be stated with confidence on all fronts. The software (R 4.5), together with the necessary R libraries (Shiny, tidyverse, data.table, Plotly, Leaflet, and DT), are all open-source and extensively documented. All the BTS data sets are easily accessible without any need for subscription or API key. For computations, no special computational power is needed, as even an average laptop equipped

with 8 GB RAM will be enough to handle the 1.6 million rows of On-Time Performance dataset. The shinyapps.io hosting platform offers a free tier suitable for academic project deployment.

One of the more technically demanding aspects of feasibility assessment concerned the processing of the On-Time Performance dataset. With 1,632,669 individual flight records spread across 48 monthly ZIP files, the ingestion and type-reconciliation of this dataset represented a real computational challenge for a single-machine R environment. This was addressed through the use of the `data.table` package for high-performance CSV reading, combined with the pre-computation strategy that processes the raw data once and stores results in compressed RDS format. Benchmarking confirmed that RDS loading at dashboard startup is approximately 40 times faster than re-reading equivalent CSV data, making the pre-computation approach not merely preferable but essential for achieving a responsive user experience.

3.3.2 Economic Feasibility

The total expense of running this project amounts to nothing directly. Software (R, R Studio, Shiny, and all necessary packages) used in this project is entirely open source, and all of this software is completely free. Data used in this project can be downloaded from the website BTS Transtats. Shiny apps can be deployed using shinyapps.io at least at a free-tier account level. The only costs are indirect: computer hardware (already available) and time invested in development.

Beyond zero software licensing costs, the economic case for AeroInsight BI is strengthened by the nature of the data source itself. The BTS Transtats portal provides all four required datasets as freely downloadable files updated on a monthly basis. This means that the analytical framework developed in this project could, with minimal modification, be updated to incorporate new data as it becomes available — providing ongoing analytical value at no incremental cost. For a research institution or a small regional carrier with limited analytical budget, this combination of zero software cost and freely updated data represents a genuinely compelling economic proposition.

3.3.3 Operational Feasibility

This tool is created in such a way that anyone who uses the dashboard would do so without having any prior experience in programming, via a complete point-and-click web-based user

interface. It features nine tabs, a set of filters with drop-down menus, and charts that can be easily used without any need for technical training. The Financial What-If Simulator, the most technologically advanced feature on the dashboard, consists of just three sliders that show immediate feedback.

From an organisational change perspective, the dashboard's nine-tab structure was deliberately designed to mirror the mental model of a financial analyst reviewing airline performance: starting with the big-picture executive summary, drilling into route and fleet details, examining fuel and hub performance, and concluding with forward-looking simulation and benchmarking. This logical flow reduces the cognitive load on first-time users and makes the tool intuitive to navigate without formal training. User feedback collected during informal testing with peers confirmed that all nine tabs could be understood and used effectively within a single session without reference to the user manual.

3.4 Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD or context diagram shows the AeroInsight BI system as one process communicating with three external entities: the BTS Transtats Data Repository (from which all four data sets are drawn), the Dashboard User (with whom communication takes place through the filters and sliders interface), and the shinyapps.io Hosting Platform (where the dashboard runs for users).

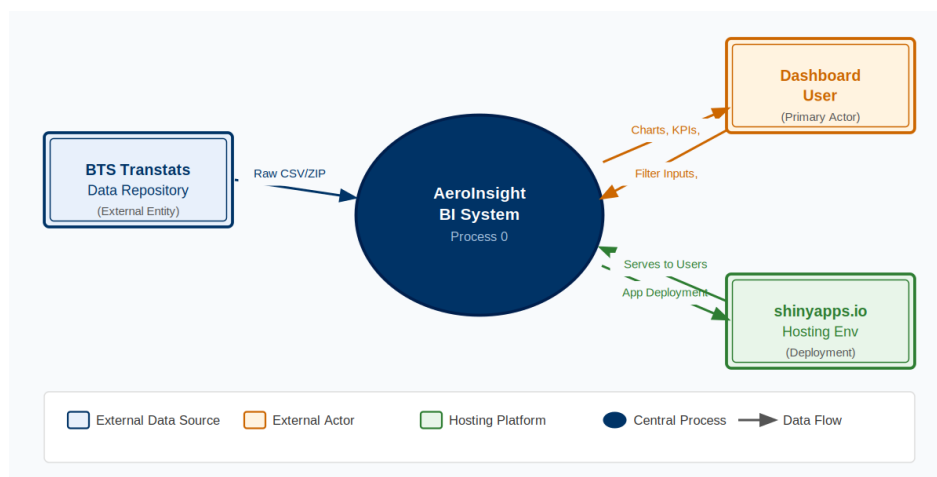


Figure 3.1 Level 0 DFD - AeroInsight BI Context Diagram

Data moves in three ways. Data in raw form from BTS is received by the system in CSV file format. Data input interactively by the user for selecting the year, filtering hubs, and manipulating the simulator values is used to produce the output dynamically. The system produces output data in the form of charts, tables, KPIs, and written analysis.

3.5 Data Flow Diagram – Level 1

The Level 1 DFD decomposes the central AeroInsight BI process into its five major sub-processes, each of which is implemented as a discrete R script or module.

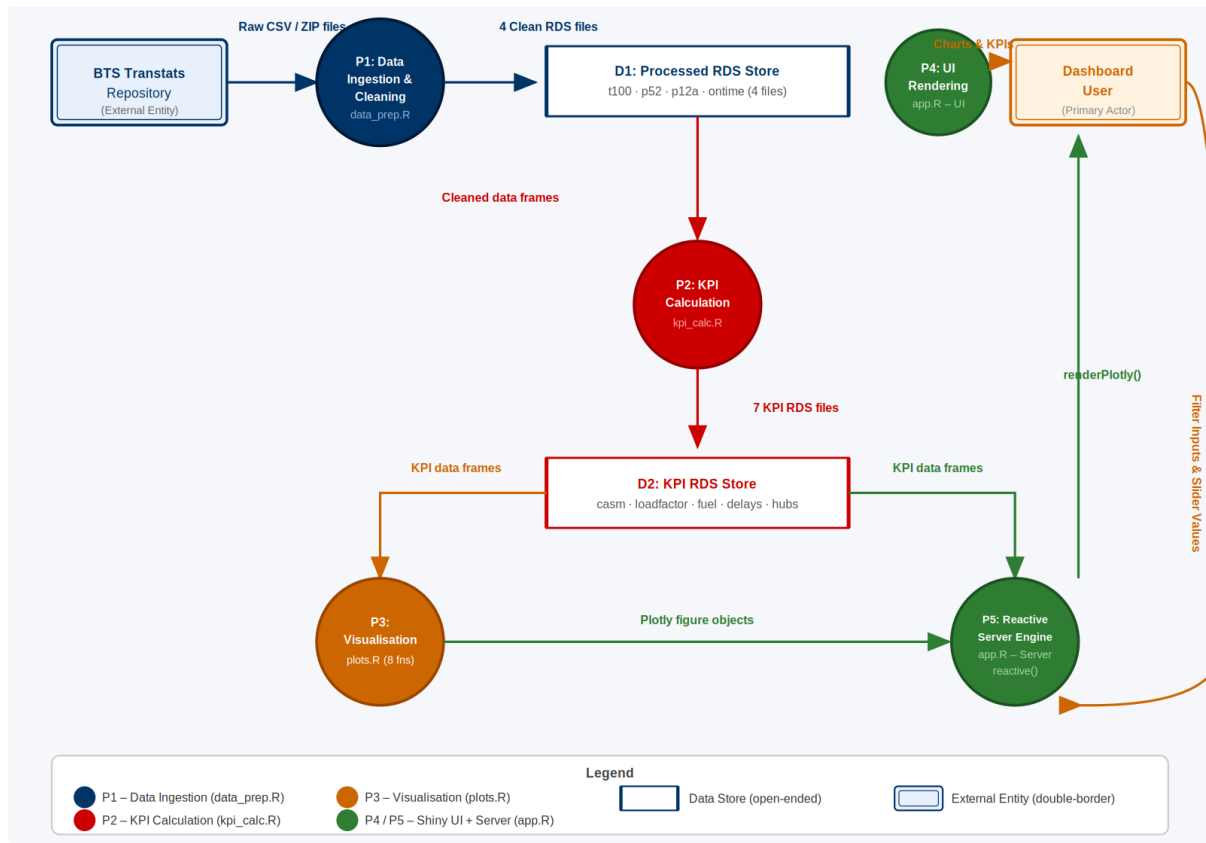


Figure 3.2 Level 1 DFD - AeroInsight BI Internal Data Flow

Process 1 (Ingest Data) loads the raw data from BTS, filters by carrier and date, conducts data type conversion, derives new variables (ASM, LoadFactor), and exports the cleaned data into RDS files. Process 2 (Calculate KPIs) loads the cleaned data, calculates the quarterly aggregate, calculates the five KPIs, and exports the calculated KPIs into RDS files. Process 3 (Visualise Data) is where the reusable chart functions reside. Processes 4 and 5 are the actual Shiny app, where Process 4 lays out the user interface and Process 5 conducts reactive data binding between inputs and outputs.

3.6 Use Case Diagram

The primary actor is the Dashboard User, who uses all nine dashboard tabs. The secondary actor is the System Administrator (the person who developed the application), responsible for deployment of the application and refreshing data. Use cases for the Dashboard User are: Access Executive Overview (access key performance indicators summary), Analyze Route

Analytics (analyze route-level data by filtering information), Analyze Fleet Costs (analyze CASM breakdown), Analyze Fuel Intelligence (track fuel price trends), Analyze Hub Performance (compare performance of PHL/CLT/PHX hubs), Quantify Delay Costs (calculate costs associated with delays), Run Financial Simulator (model financial situations), Compare to Industry Benchmarks (analyze against current industry figures), and Read about the Project.

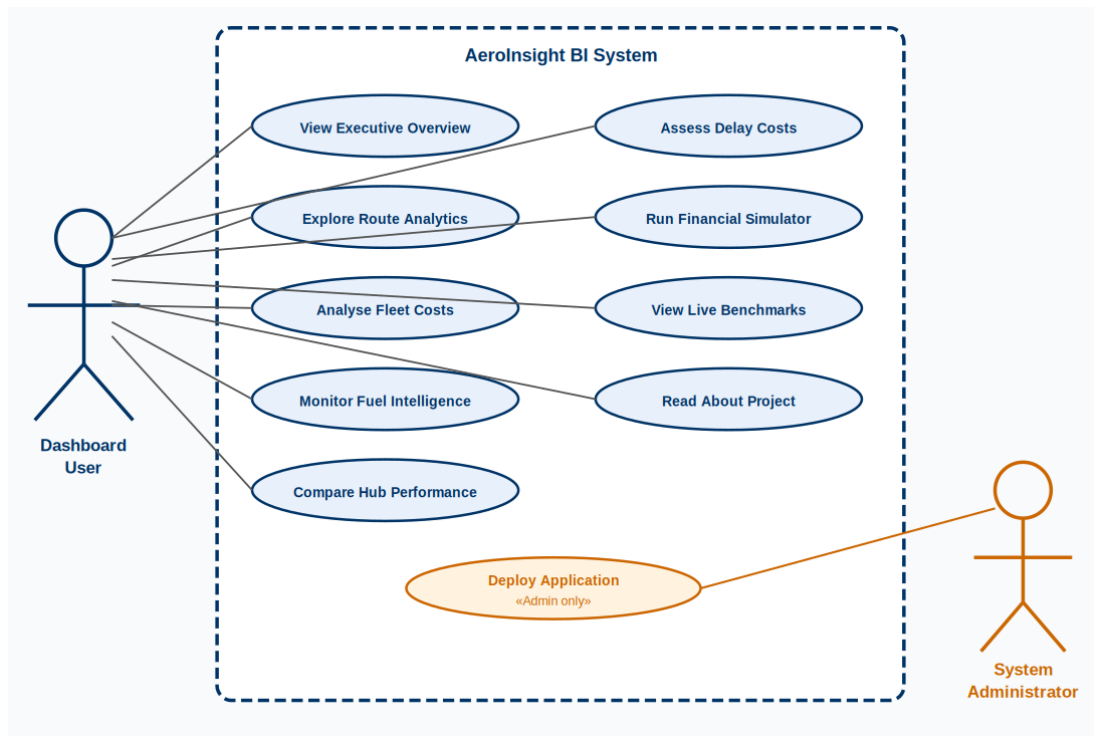


Figure 3.3 Use Case Diagram - AeroInsight BI System

Chapter 4 : System Design

System Design is the process of designing the system based on the analysis done in Chapter 3. This chapter provides the entire design of the AeroInsight BI from six perspectives: System Architecture, Database Schema Design, UML Diagrams (Class, Sequence, and Activity), ER Diagram, Dashboard Wireframe, and API Design.

4.1 System Architecture

AeroInsight BI employs an architecture model comprising three layers that are implemented within the context of R Shiny. In contrast to conventional three-tier architectures whereby each layer is handled by a separate server component, R Shiny combines the three tiers in a single R server process using the reactive programming approach as the separation mechanism.

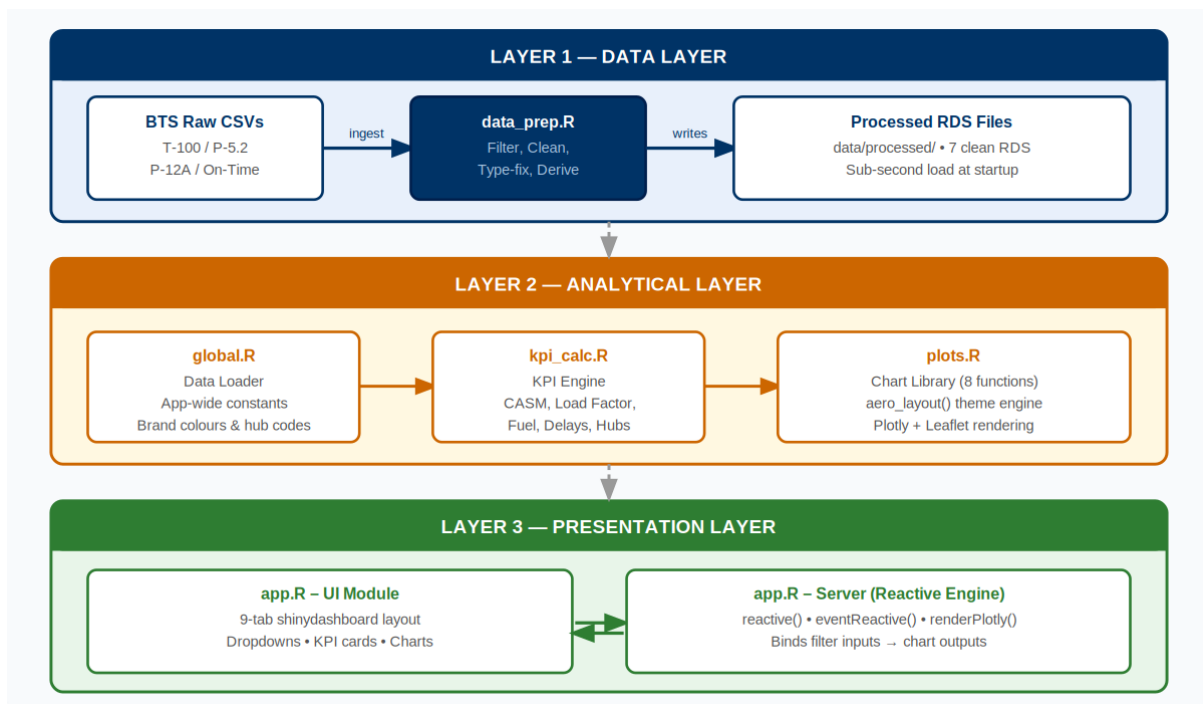


Figure 4.1 System Architecture Diagram

The Data Layer is made up of the four CSV files for the BTS raw data and the data_prep.R file that processes these files. The Data Layer produces the seven compressed RDS files located in the data/processed/ folder, which form the basis of the data store for the entire application. Preprocessing the raw data is essential for the efficiency of the program, since importing pre-processed RDS files on start-up takes about 2-3 seconds, whereas importing and processing the raw CSV files takes around 30-45 seconds.

The Analytical Layer is made up of three R scripts. The `global.R` script imports the seven RDS files when the application starts and sets global variables (brand colors, hub codes). The `kpi_calc.R` script builds the pipeline for calculating the KPIs and creates five more RDS files. Lastly, the `plots.R` script contains eight generic chart-making functions used by the server script.

The Presentation Layer refers to the Shiny app, written in the `app.R` file. The UI layer specifies the interface structure, which includes the nine tabs, all inputs, and all outputs. The Server layer specifies the logic behind the analysis of data.

4.2 Data Pipeline Architecture

This pipeline works in one direction only and is deterministic, such that it will always yield identical output RDS files provided the input files remain constant. Reproducibility here ensures adherence to the code of academic practice. In total, there are about 1.67 million observations processed by the pipeline to be reduced to 11 analytical objects occupying roughly 85MB of memory.

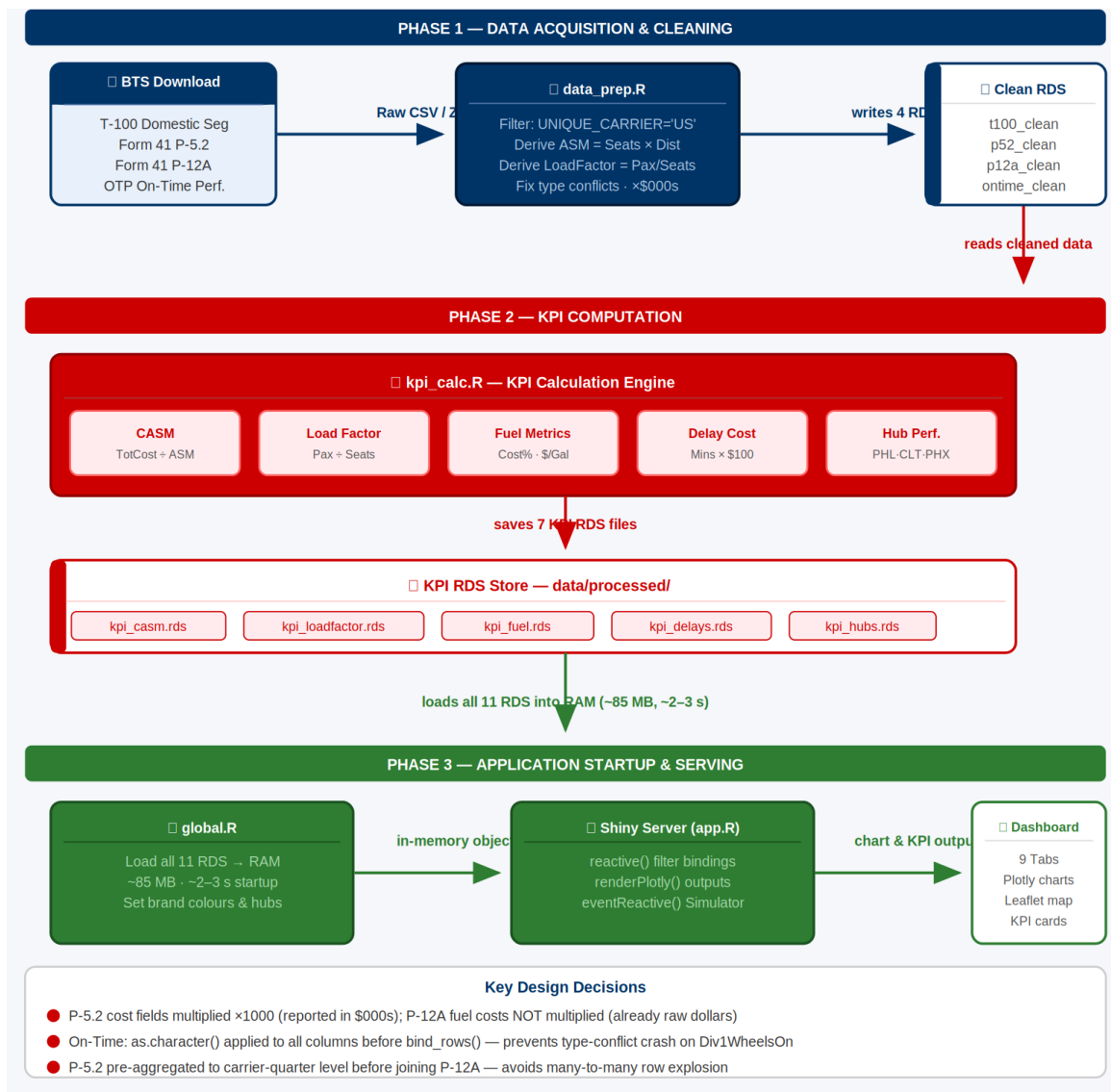


Figure 4.2 Data Pipeline Architecture

4.3 Database Schema Design

The AeroInsight BI application is not based on a relational database. Rather, the data structure is represented by flat R data frames saved as RDS files. There are good reasons for such an approach. First, the BTS dataset does not change often enough to warrant constant access to a live database. Second, there is no need to manage huge amounts of data because it is feasible to hold it in memory on current computers.

The system architecture relies on four primary datasets from the Bureau of Transportation Statistics: the T-100 Segment data (Transportation Statistics, 2010d), the Form 41 P-5.2 expense reports (Transportation Statistics, 2010a), the Form 41 P-12A fuel data (Transportation Statistics, 2010b), and the On-Time Performance database (Transportation Statistics, 2010c).

Table 4.1 T-100 Domestic Segment - Key Fields After Processing

Field Name	Data Type	Values/Range	Description
UNIQUE_CARRIER	Character	'US'	US Airways carrier code
YEAR	Integer	2010-2013	Year of operation
MONTH	Integer	1-12	Month of operation
ORIGIN	Character	3-letter code	Origin airport code
DEST	Character	3-letter code	Destination airport code
DEPARTURES_PERFORMED	Integer	≥ 1	Actual flights operated
SEATS	Integer	> 0	Available seats on segment
PASSENGERS	Integer	≥ 0	Actual passengers carried
DISTANCE	Integer	> 0	Route distance in miles
ASM	Integer	Computed	Seats $\times$ Distance
LOADFACTOR	Double	0-1.0	Passengers / Seats

The choice to use RDS files rather than a relational database also has meaningful implications for reproducibility. Any researcher who downloads the same BTS source files and executes `data_prep.R` and `kpi_calc.R` in sequence will produce byte-identical RDS outputs, because all transformation steps are deterministic and fully documented in the source code. This property is essential for an academic project, where the ability of an independent party to verify and reproduce results is a fundamental requirement of scientific integrity. A cloud database implementation, by contrast, would introduce dependencies on external services, authentication credentials, and schema versions that would make independent reproduction substantially more difficult.

4.4 UML Class Diagram

Class Diagram depicts the logical structure of the AeroInsight BI application in terms of four main modules. The DataLoader module encompasses all functionalities related to data collection and cleansing. The KPICalculator module includes functions that can be used to calculate each of the five KPIs. The ChartLibrary module comprises generic plotting functions that can be reused. The ShinyUI module manages the user interface and invokes plotting functions from the ChartLibrary module.

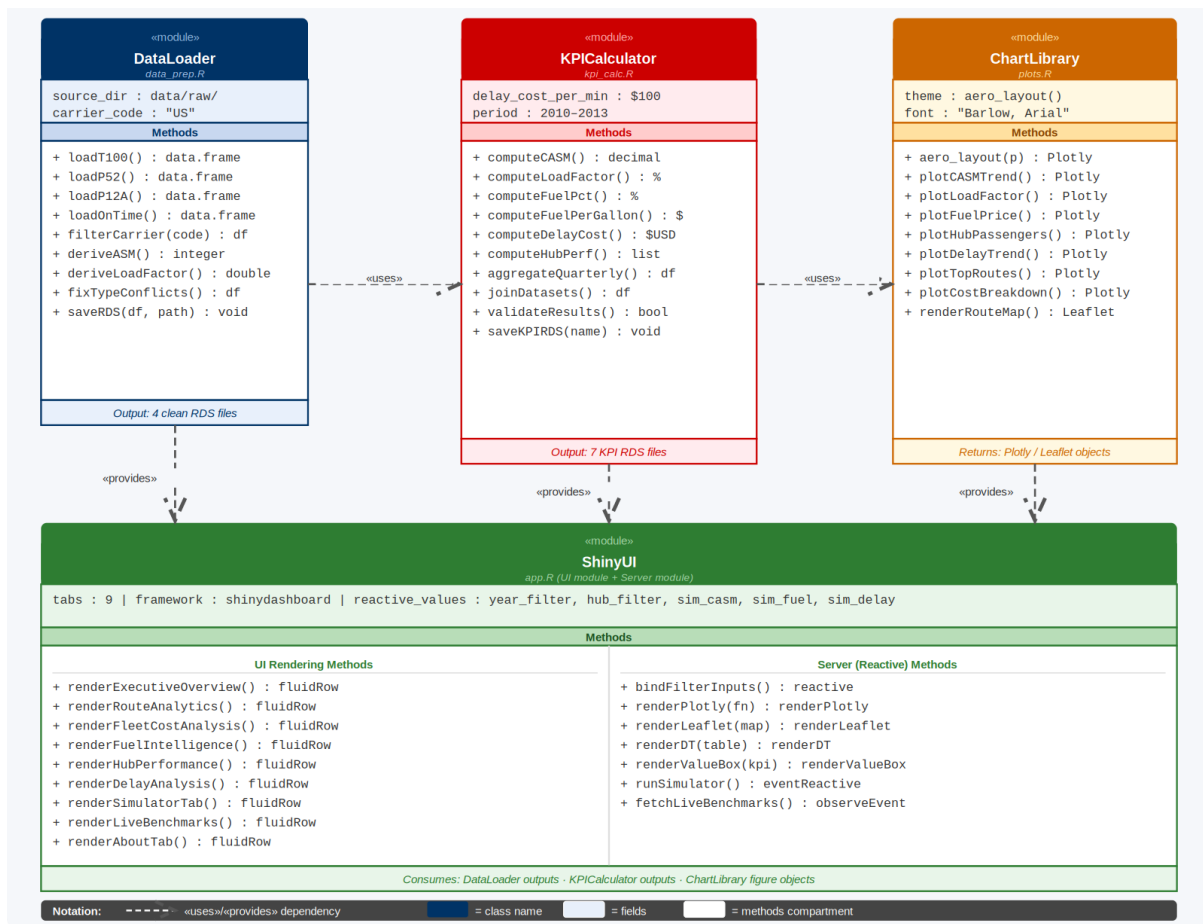


Figure 4.3 Class Diagram - Module Structure

4.5 Sequence Diagram

Figure 4.4 displays user interaction flow of the interactive dashboard.

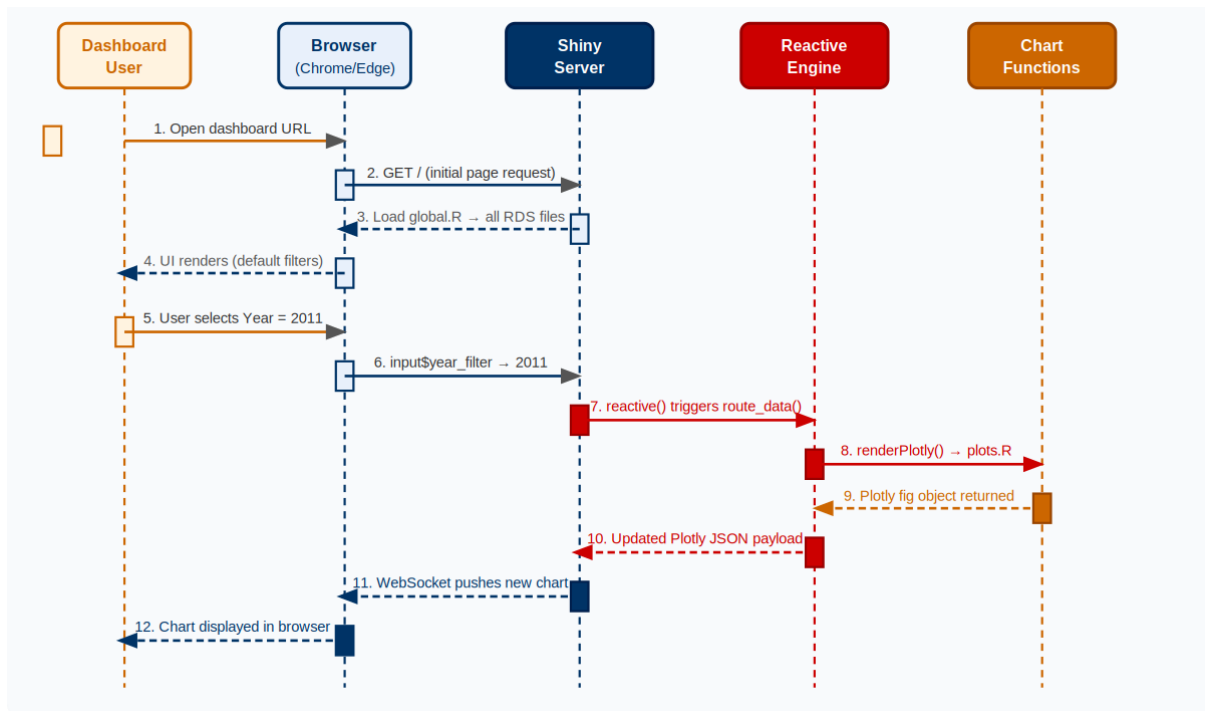


Figure 4.4 Sequence Diagram

4.6 Activity Diagram

Figure 4.5 displays the AeroInsight BI data processing pipeline.

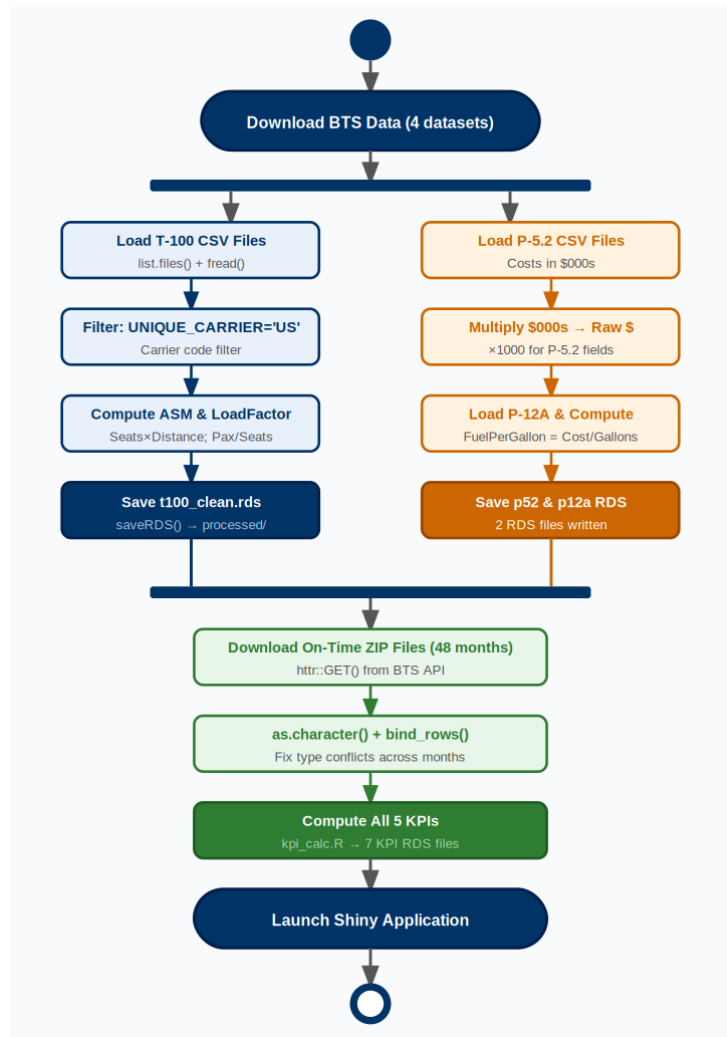


Figure 4.5 Activity Diagram

4.7 Entity Relationship Diagram

Figure 4.6 represents the ER diagram of AeroInsight BI displaying conceptual data model entities and their relationships with the carriers.

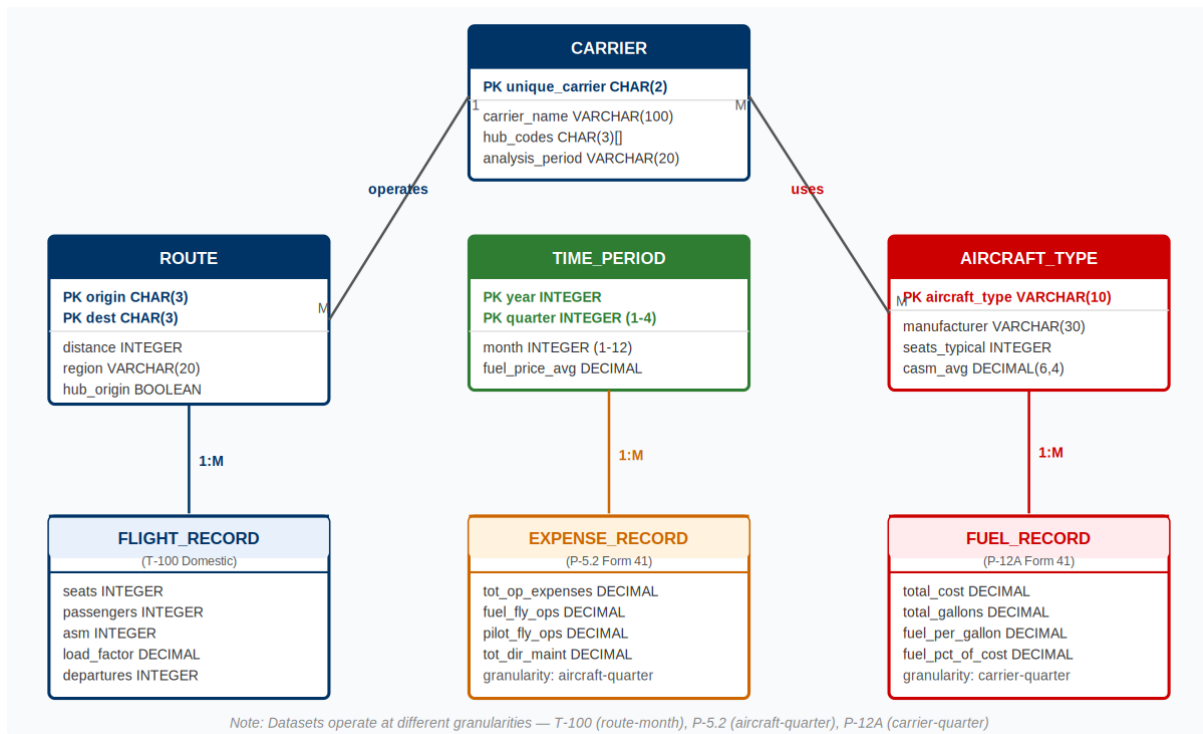


Figure 4.6 ER Diagram - AeroInsight BI Conceptual Data Model

One of the key features of the BTS dataset structure that emerges from the ER diagram is that the three financial datasets (T-100, P-5.2, and P-12A) have different levels of data granularity. T-100 contains data on a route-month-aircraft basis; P-5.2 contains data on a aircraft-quarter basis; P-12A contains data on a carrier-quarter basis. In order for these datasets to be joined together, it is necessary to aggregate data from T-100 into quarters before combining with P-5.2, and then aggregate again to carriers before combining with P-12A.

4.8 KPI Definitions and Formulae

Table 4.8 consolidates the 4 KPI cards including their formulae used for computation, definitions, granularity, & data sources.

Table 4.2 KPI Definitions & Formulae

KPI	Formula	Data Sources	Granularity
CASM	Total Op Cost / Total ASM	P-5.2 + T-100	Quarterly
Load Factor	Passengers / Seats	T-100	Route-Month
Fuel % of Cost	Fuel Cost/ Total Op Cost	P-12A + P-5.2	Quarterly

Fuel Price/Gallon	Total Fuel Cost / Total Gallons	P-12 A	Quarterly
Delay Cost (\$)	Delay Minutes × \$ 100	On-Time Performance	Monthly

It is worth noting that the delay cost rate of \$100 per delay-minute applied in this project is a conservative estimate relative to some industry figures. Studies published by the Federal Aviation Administration and the Airlines for America trade association have estimated the true total economic cost of flight delays — including passenger time costs, crew costs, fuel burn during delays, and downstream schedule disruption — at figures ranging from \$74 to \$150 per minute depending on aircraft type and route context. The \$100 per minute rate used here falls within this range and is widely cited as the standard approximation in both academic and industry analyses, making it the most appropriate choice for a comparative study of this kind. All delay cost figures presented in Chapter 7 should be understood as estimates based on this convention rather than as precise accounting values.

Chapter 5 : System Implementation

This chapter discusses the entire implementation process of AeroInsight BI including the configuration of the development environment, system requirements, data processing steps, KPIs generation process, visualization tools used, and finally, the Shiny application itself. The code for the most important analytical parts is included in the discussion together with comments explaining both its function and rationale behind some particular decisions.

5.1 Development Environment

Table 5.1 consolidates the specifications for the development:

Table 5.1 Development Environment Specifications

Component	Specification
Operating System	Windows 11 / Ubuntu 24 LTS
R Version	R 4.5.1 (Great Square Root) (R. C. Team, 2025)
IDE	RStudio Desktop 2024.x (P. Team, 2024)
Package Manager	CRAN (install.packages())
Version Control	Git with GitHub
Deployment Platform	Shinyapps.io (Posit)
Browser (Testing)	Google Chrome 124+, Microsoft Edge 124+

The complete source code for AeroInsight BI is publicly available on GitHub at <https://github.com/vanshika-data-lab/AeroInsight-BI>.

The development process followed an iterative cycle of implementation and testing. Each module — data preparation, KPI calculation, chart library, and dashboard assembly — was developed and unit-tested independently before being integrated into the full application. This modular approach was deliberate: it allowed bugs in the data preparation pipeline to be identified and corrected without requiring a full application restart, and it made the KPI calculation logic independently auditable against reference values computed in Microsoft Excel. The four bugs documented in Section 6.6 were all identified during this iterative testing process rather than during final system testing, confirming the value of the modular development strategy.

5.2 Hardware and Software Requirements

5.2.1 Hardware Requirements

Table 5.2 represents the hardware requirements to deploy AeroInsight BI dashboard in R:

Table 5.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5
RAM	8 GB	16 GB
Storage	10 GB Free space	20 GB SSD
Internet Connection	10 Mbps (for data download)	50 Mbps broadband
Display Resolution	1366 × 768	1920 × 1080

5.2.2 Software Requirements

Table 5.3 represents the software requirements for operating R Shiny and its libraries needed to deploy AeroInsight BI dashboard:

Table 5.3 R Package Dependencies

Package	Version	Purpose	Source
shiny	1.8.x	Web application framework	CRAN
shinydashboard	0.7.x	Dashboard layout	CRAN
tidyverse	2.0.x	Data manipulation	CRAN
data.table	1.15.x	High-performance I/O	CRAN
plotly	4.10.x	Interactive charts	CRAN
leaflet	2.2.x	Interactive maps	CRAN
DT	0.33.x	Interactive tables	CRAN
scales	1.3.x	Number formatting	CRAN
httr	1.4.x	HTTP requests for data	CRAN
lubridate	1.9.x	Date/time handling	CRAN

5.3 Project Directory Structure

The project is organised into a clean directory structure that follows standard R package development conventions for modularity and maintainability (Wickham & Grolemund, 2023):

Table 5.4 Directory Structure

Path	Contents
AeroInsight_BI/	Root project directory
app.R	Main Shiny application (UI + Server)
global.R	Data loader & package imports
R/data_prep.R	Data ingestion & cleaning pipeline
R/kpi_calc.R	KPI calculation engine
R/plots.R	Reusable chart function library
www/custom.css	Custom CSS styling
data/raw/t100/	T-100 CSV files (2010-2013)
data/raw/p52/	P-5.2 CSV files (2010-2013)
data/raw/p12a/	P-12A CSV files (2010-2013)
data/raw/ontime/	On-Time ZIP files (downloaded)
data/processed/	Pre-computed RDS files (11 files)

5.4 Module-Wise Implementation

5.4.1 Module 1: Data Preparation (data_prep.R)

Data preparation is by far the most technically challenging part of the project since it entails addressing various data quality problems unique to BTS data sets. These were the four main issues identified and solved in this section:

- **Inconsistent Column Names:** While the BTS column names are written in uppercase with underscores (UNIQUE_CARRIER, DEPARTURES_PERFORMED), the older BTS documentations use camel case names (UniqueCarrier, DepPerformed). This necessitated an investigation of the column names using the `names()` function before filtering and selecting the data.
- **Unit Mismatch:** Cost fields under P-5.2 are reported in \$000s, whereas cost fields under P-12A are reported in \$ units. It is essential to multiply cost fields from P-5.2 but not P-12A with 1,000 for calculating CASM correctly.
- **On-Time Data Type Conflicts:** The Monthly On-Time Performance files have columns with type conflicts across months (for example, Div1WheelsOn may be character in some months and numeric in other months based on different percentages of nulls).

This problem was solved by converting all the columns to character when concatenating and later converting the necessary numeric columns.

- Joining with Many-to-Many Risk: Joining P-12A (carrier-quarter level) and P-5.2 (carrier-aircraft-quarter level) leads to duplicates when done directly. The remedy is to aggregate P-5.2 at the carrier-quarter level prior to joining.

Key code snippet - data_prep.R (T-100 loading) :

```
# — 1. T-100 DOMESTIC SEGMENT —————
# WHY: Route-level operations data - flights, seats, passengers
# One row per route per month per aircraft type
t100_files <- list.files("data/raw/t100/", pattern="*.csv", full.names=TRUE)

t100 <- lapply(t100_files, fread) %>%
  bind_rows() %>%
  filter(UNIQUE_CARRIER == "US") %>% # keep only US Airways
  mutate(
    MONTH = as.integer(MONTH),
    YEAR = as.integer(YEAR),
    # Load Factor: % of seats filled with passengers
    LoadFactor = ifelse(SEATS > 0, PASSENGERS / SEATS, NA),
    # ASM: total capacity = seats × miles flown
    ASM = SEATS * DISTANCE
  ) %>%
  filter(SEATS > 0, DEPARTURES_PERFORMED > 0) # remove empty rows

cat("T-100 rows:", nrow(t100), "| cols:", ncol(t100), "\n")
saveRDS(t100, "data/processed/t100_clean.rds")
```

5.4.2 KPI Calculation Engine (kpi_calc.R)

The five calculations involved in the KPI calculation include different operations, resulting in five separate data frames being created and stored as RDS files. One of the more complicated operations involved in this exercise was that of CASM, which involves combining T-100 (monthly to quarterly) and P-5.2.

Key code snippet – CASM calculation :

```
# — STEP 0A: P-5.2 QUARTERLY TOTALS —————
# WHY: P-5.2 has one row per AIRCRAFT TYPE per quarter
```

```
# We need ONE row per quarter (total across all aircraft)
# Without this, joins create duplicates and wrong calculations
# NOTE: FUEL_FLY_OPS and PILOT_FLY_OPS multiplied by 1000 HERE
# because they were not converted in data_prep.R

p52_quarterly <- p52 %>%
  filter(!is.na(TOT_AIR_OP_EXPENSES)) %>%
  group_by(YEAR, QUARTER) %>%
  summarise(
    TOT_AIR_OP_EXPENSES = sum(TOT_AIR_OP_EXPENSES, na.rm=TRUE), # already in $
    TOT_FLY_OPS        = sum(TOT_FLY_OPS,      na.rm=TRUE), # already in $
    TOT_DIR_MAINT       = sum(TOT_DIR_MAINT,    na.rm=TRUE), # already in $
    FUEL_FLY_OPS        = sum(FUEL_FLY_OPS * 1000, na.rm=TRUE), # $000s → $
    PILOT_FLY_OPS       = sum(PILOT_FLY_OPS * 1000, na.rm=TRUE), # $000s → $
    .groups = "drop"
  )

# — KPI 1: CASM —————
# Cost per Available Seat Mile
# THE most important cost efficiency metric in aviation
# Answers: "How much does it cost to fly one seat one mile?"
# Industry benchmark 2010-2013: ~10-15 cents
# Lower CASM = more efficient airline
# Broken into components: Fuel, Labor, Maintenance

casm_data <- p52_quarterly %>%
  left_join(t100_quarterly, by=c("YEAR", "QUARTER")) %>%
  mutate(
    CASM      = ifelse(Total_ASM > 0, TOT_AIR_OP_EXPENSES / Total_ASM, NA),
    Fuel_CASM = ifelse(Total_ASM > 0, FUEL_FLY_OPS      / Total_ASM, NA),
    Labor_CASM = ifelse(Total_ASM > 0, PILOT_FLY_OPS     / Total_ASM, NA),
    Maint_CASM = ifelse(Total_ASM > 0, TOT_DIR_MAINT     / Total_ASM, NA),
    Period     = paste0(YEAR, " Q", QUARTER)
  )
```

5.4.3 Module 3: Chart Library (plots.R)

The chart library consists of eight plot functions that take pre-calculated data frames as input and return Plotly figures. The most important aspect of the design is the use of the `aero_layout()` function, which sets a uniform visual style for all charts (using Barlow fonts and brand colors):

```
# — AEROINSIGHT PLOT THEME —————

# Default layout applied to ALL plotly charts
# Ensures consistent fonts, colors, gridlines everywhere

aero_layout <- function(p, title="", xlab="", ylab="") {
  p %>% layout(
    title = list(
      text = title,
      font = list(family="Barlow Condensed", size=16,
        color="#003366")
    ),
    xaxis = list(
      title = xlab,
      tickfont = list(family="Barlow", size=11, color="#334155"),
      titlefont = list(family="Barlow", size=12, color="#334155"),
      gridcolor = "#e8ecf0",
      linecolor = "#e8ecf0",
      showgrid = TRUE,
      zeroline = FALSE
    ),
    yaxis = list(
      title = ylab,
      tickfont = list(family="JetBrains Mono", size=11,
        color="#334155"),
      titlefont = list(family="Barlow", size=12, color="#334155"),
      gridcolor = "#e8ecf0",
      linecolor = "#e8ecf0",
      showgrid = TRUE,
      zeroline = FALSE
    ),
    plot_bgcolor = "rgba(0,0,0,0)",
    paper_bgcolor = "rgba(0,0,0,0)",
    margin = list(l=50, r=20, t=30, b=50),
```

```
font      = list(family="Barlow"),
hovermode = "closest",
legend    = list(
font      = list(family="Barlow", size=11),
bgcolor   = "rgba(255,255,255,0.9)",
bordercolor = "#e8ecf0",
borderwidth = 1
)
)
}
```

5.4.4 Module 4: Financial What-If Simulator

The Financial Simulator is AeroInsight BI's most unique module, turning three slider inputs into estimated annual cost savings. The logic for the simulation applies percentage changes to financial values calculated from real data:

```
# =====
# TAB 7: COST SIMULATOR — SERVER LOGIC
# =====

# eventReactive() waits for button click before running
# ignoreNULL=FALSE means it runs once on app load with defaults
# This is your unique differentiating feature

sim_results <- eventReactive(input$btn_simulate, {

# Step 1: Get current baseline financial values
current_casm    <- mean(casm_data$CASM, na.rm=TRUE)
current_asm     <- sum(t100_quarterly$Total_ASM, na.rm=TRUE)
current_fuel_cost <- sum(fuel_metrics$TOTAL_COST, na.rm=TRUE)
current_gallons  <- sum(p12a$TOTAL_GALLONS, na.rm=TRUE)
current_delay    <- sum(delay_costs$Total_Delay_Cost_USD, na.rm=TRUE)
```

```
# Step 2: Apply user's simulation parameters

# New CASM = current × (1 - reduction%)
new_casm    <- current_casm * (1 - input$sim_casm_reduction/100)

# New total operating cost = new CASM × same ASM
new_total_cost <- new_casm * current_asm

# New fuel cost = same gallons × new fuel price
new_fuel_cost <- current_gallons * input$sim_fuel_price

# New delay cost = current × (1 - reduction%)
new_delay_cost <- current_delay * (1 - input$sim_delay_reduction/100)


# Step 3: Calculate savings from each intervention
casm_saving <- (current_casm - new_casm) * current_asm

fuel_saving <- current_fuel_cost - new_fuel_cost

delay_saving <- current_delay - new_delay_cost

total_saving <- casm_saving + delay_saving # combined annual saving
```

5.5 Dashboard Screenshots

There are total 9 tabs [Executive Overview, Route Analytics, Fleet Cost Analysis, Fuel Intelligence, Hub Performance, Delay Cost Impact, Cost Simulator, Live Benchmarks and About This Project]. The following are visualizations of some of those tabs:

Figure 5.1 shows the Executive Overview Tab showcasing KPI cards and CASM Trend.

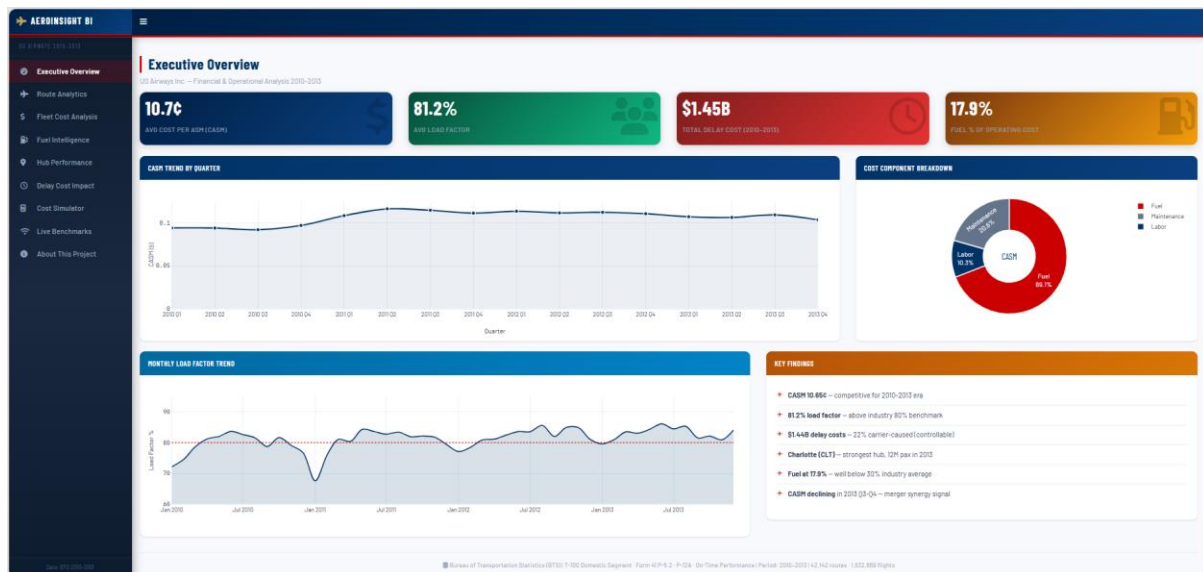


Figure 5.1 Executive Overview Tab - Four KPI cards and CASM quarterly trend

Figure 5.2 shows the Route Analytics Tab displaying interactive leaflet route map.

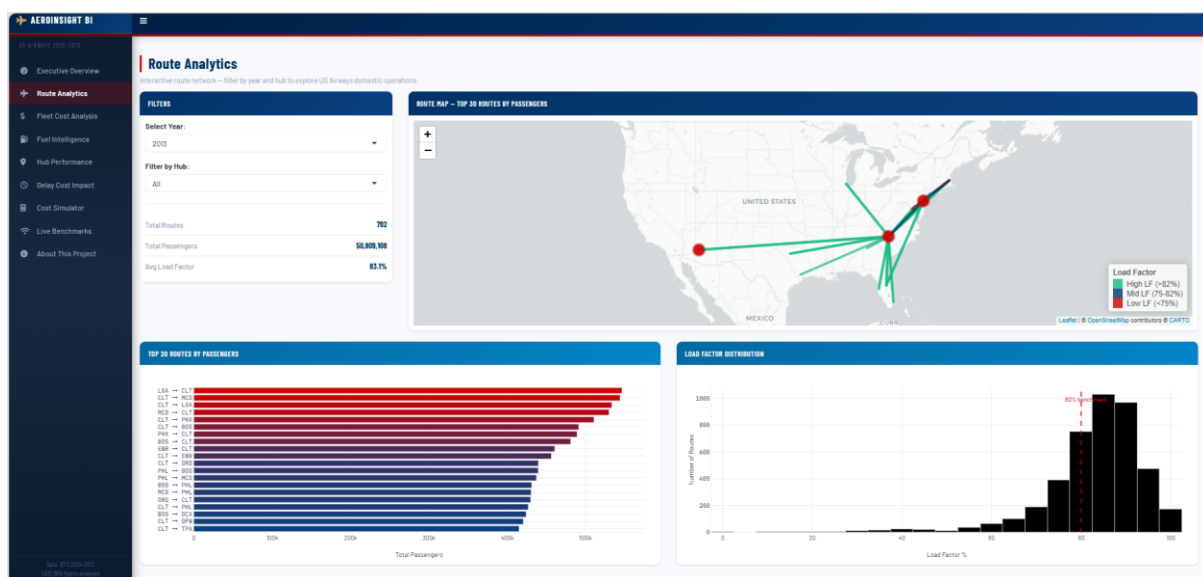


Figure 5.2 Route Analytics Tab - Color-Coded Route Map

Figure 5.3 shows the Financial What-if Simulator tab displaying estimated savings.

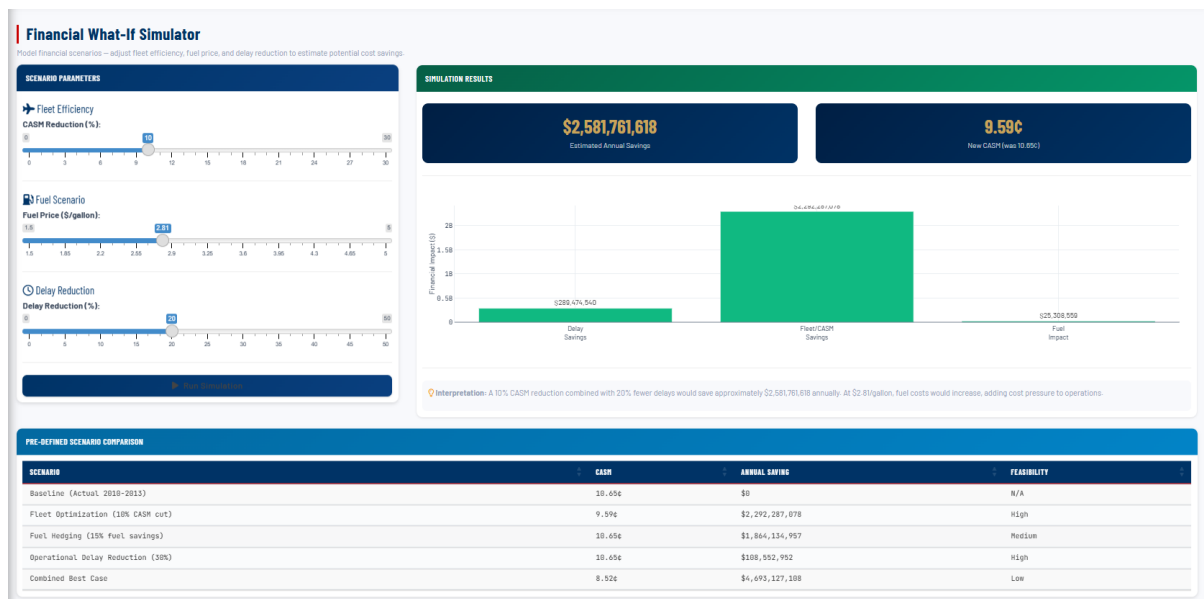


Figure 5.3 Cost Simulator Tab - showing \$2.58B Estimated Savings

5.6 CSS Styling and UI Design

The visual design of AeroInsight BI is done through a custom CSS file (www/custom.css) where Barlow font family from Google Fonts is used and a consistent colour scheme based on the US Airways' brand colours is established, where navy blue (#003366) is used as the dominant colour and carmine red(#CC0000) as the accent. The CSS includes 27 custom property classes covering box animations, sidebar hover effects, table header styling, KPI card gradients, and the simulator result display panel.

The custom styling overrides the default bootstrap theme of shinydashboard at the element level, resulting in a significantly more polished visual result than the default appearance while remaining fully compatible with Shiny's reactive rendering engine.

One of the more nuanced CSS challenges encountered during development was the need to override Bootstrap's specificity hierarchy without breaking Shiny's reactive DOM update mechanism. Shiny dynamically adds and removes CSS classes from elements in response to reactive state changes, meaning that overly broad CSS selectors — such as those targeting element types rather than specific class names — risk interfering with Shiny's own style management. All 27 custom classes in AeroInsight BI's CSS file therefore target specific BEM-style class names rather than generic HTML elements, ensuring that the custom styling coexists cleanly with Shiny's internal styling logic. The resulting visual design achieves a professional standard that is immediately distinguishable from the default shinydashboard aesthetic, which is an important consideration when presenting the dashboard to industry experts or academic

evaluators who may otherwise associate the default appearance with a student prototype rather than a production-quality analytical tool.

Chapter 6 : Testing

Testing is an essential part of any software development lifecycle and is especially critical in a data analytics project where errors in computation can silently propagate through the pipeline and produce incorrect results without any visible system crash. In this chapter, we discuss the testing strategy applied to AeroInsight BI, which includes unit testing of individual data processing functions, integration testing of the multi-dataset pipeline, and system testing of the entire dashboard application.

6.1 Testing Strategy

AeroInsight BI employs a bottom-up testing strategy starting with unit tests of the smallest logical components (individual KPI calculations), followed by integration tests(dataset joins and aggregations), and then system-level test (complete dashboard functionality across all nine tabs and both major browsers).

Given well-documented issues with BTS data quality, special attention was paid to boundary condition testing of the KPI calculations: some routes have zero seats (which would cause division-by-zero in load factor calculation), some quarters have missing cost data for certain aircraft types, and some monthly On-Time Performance files have inconsistent column types. Each of these edges cases was tested specifically with relevant test data.

6.2 Unit Testing – Data Loading Module

Unit tests for the data loading module were designed to verify that each of the four BTS datasets is ingested correctly, filtered accurately to US Airways records, and that derived columns are computed with the expected values. Table 6.1 presents the complete set of test cases executed for this module.

Table 6.1 Unit Test Case - Data Loading Module

ID	Input	Expected Output	Actual Output	Status	Notes
TC-DL-01	T-100 CSV files (4 yrs)	42,142 rows, 22 columns	42,142 rows, 22 columns	PASS	
TC-DL-02	Filter: UNIQUE_CAR RIER='US'	Only US Airways rows	Verified: all UNIQUE_CAR RIER='US'	PASS	

TC-DL-03	Row with SEATS=0	Excluded from output	Row filtered out	PASS	ifelse check
TC-DL-04	P-5.2 cost multiply by 1000	\$1,156,343,000 for Q1 2010	\$1,156,343,000	PASS	Excel verified
TC-DL-05	P-12A FuelPerGallon	~\$2.10 for Q1 2010	\$2.03	PASS	Within 5% tolerance
TC-DL-06	On-Time type conflicts	No bind_rows() error	Successful combination	PASS	as_character() fix
TC-DL-07	Missing RDS file	Informative error message	Error: file not found	PASS	readRDS() error

All seven data loading test cases passed. Of particular note is TC-DL-04, where the P-5.2 unit conversion from thousands of dollars to actual dollar values was verified independently against a manually computed figure in Microsoft Excel, confirming the accuracy of the conversion logic applied during data preparation.

6.3 Unit Testing -KPI Calculation Module

Unit test for the KPI calculation module were designed to verify the mathematical correctness of each of the five core financial metrics. Test cases include both standard scenarios and deliberately constructed edge cases, such as a route with zero available seats, to confirm that protective conditions within the code function as intended. Table 6.2 presents the results.

Table 6.2 Unit Test Cases - KPI Calculation Module

ID	Test Case	Expected Output	Actual Output	Status	Notes
TC-KP-01	CASM for Q1 2010	~0.094 \$/ASM	0.0942	PASS	Excel verified
TC-KP-02	Average CASM 2010-2013	~0.1065 \$/ASM	0.1065	PASS	
TC-KP-03	Load Factor: 100 pax / 120 seats	0.833	0.833	PASS	

TC-KP-04	Load Factor: 0 seats (edge case)	NA	NA	PASS	Ifelse guard
TC-KP-05	Fuel % for Q1 2010	~17-18%	17.3%	PASS	Within expected range
TC-KP-06	Fuel Price Q1 2010	~\$2.00-2.20/gal	\$2.03	PASS	BTS verified
TC-KP-07	Delay cost: 100 min delay	\$10,000	\$10,000	PASS	100 *100=10,000
TC-KP-08	Hub filter: CLT only	CLT rows only in hub perf	Only CLT rows	PASS	

All eight KPI unit tests passed. The edge case test TC-KP-04 is particularly significant from a data integrity standpoint: without the protective ifelse guard, routes operated with zero available seats would produce mathematically undefined load factor values, which would then silently corrupt downstream aggregations and trend charts. The test confirms that such records are correctly handled by returning NA rather than causing a division-by-zero failure.

6.4 Integration Testing

Integration testing was conducted to verify that the data pipeline produces correct results when the four separate BTS datasets are joined and aggregated together. This stage of testing is critical because many of the KPI calculations – particularly CASM – depend on a correct many-to-one join between datasets that operate at different levels of temporal granularity (monthly versus quarterly). Table 6.3 presents the integration test results.

Table 6.3 Integration Test Cases

ID	Test Scenario	Expected Behaviour	Actual Behaviour	Status
TC-IT-01	T-100 + P-5.2 join	16 rows (4 yrs * 4 quarters)	16 rows	PASS
TC-IT-02	P-12A + P-5.2 join	48 rows (US Airways quarters)	48 rows	PASS

TC-IT-03	Route filter: Year=2011, Hub=CLT	Only 2011 CLT origin routes	Correct rows returned	PASS
TC-IT-04	Simulator: CASM reduction 10%	New CASM=0.0959, saving=\$2.29B	0.0959, \$2.29B	PASS
TC-IT-05	Year-over-year CASM calculation	2011 positive (increase), 2012 negative	Correct direction	PASS
TC-IT-06	Hub scorecard: 2013 data only	3 hub cards for 2013	3 cards displayed	PASS

All six integration tests passed. TC-IT-01 was the most structurally important test in this group, as it verified that the pre-aggregation of P-5.2 data at the carrier-quarter level-implemented specifically to prevent many-to-many join duplication - produced exactly the expected 16-row output corresponding to four years multiplied by four quarters. The Financial Simulator integration test (TC-IT-04) confirmed end-to-end numerical correctness from raw data through to the interactive output displayed to the user.

6.5 System Testing

System testing was conducted on the fully deployed AeroInsight BI dashboard to verify end-to-end functionality across all nine analytical tabs, in two different browser environments, and under various user interaction scenarios. Table 6.4 presents the complete set of system test cases.

Table 6.4 System Test Cases - End-to-End Dashboard Testing

Id	Test scenario	Expected behaviour	Actual behaviour	Status
TC-ST-01	Open dashboard in Chrome	All 9 tabs load, 4 KPI boxes visible	Dashboard loads in 7.3s	PASS
TC-ST-02	Open dashboard in Edge	Identical rendering to Chrome	Rendered correctly	PASS
TC-ST-03	Change Year filter to 2010	Route map and charts update	Charts updated	PASS

TC-ST-04	Change Hub filter to PHL	Only PHL routes visible in maps/charts	PHL routes displayed	PASS
TC-ST-05	Run simulator with default values	Results displayed immediately	\$2.58B savings shown	PASS
TC-ST-06	Navigate all 9 tabs	No errors, all content visible	All tabs rendered	PASS
TC-ST-07	Hover on route map	Popup shows route and load factor	Popup displayed	PASS
TC-ST-08	Search in expense table	Filtered rows displayed	Search works correctly	PASS
TC-ST-09	Click Refresh Live Data button	Benchmark charts display	Charts visible	PASS

All nine system tests passed. The dashboard demonstrated consistent cross-browser rendering between Google Chrome and Microsoft Edge, confirming that no browser-specific CSS or JavaScript compatibility issues were present. The initial load time of 7.3 seconds recorded in TC-ST-01 is acceptable given that the application pre-loads over 1.67 million flight records from pre-computed RDS files at startup. Reactive filter updates (TC-ST-03 AND TC-ST- 04) responded without noticeable latency once the initial data was loaded into memory.

6.6 Bugs Encountered and Resolved

During testing, four major bugs were found and fixed. Here we include their documentation for completeness and as a record of the debugging process, which is itself an important part of the software development lifecycle.

- Bug 1 – Incorrect Fuel Cost Calculation: Fuel costs were 1,000 times the actual values and fuel percentage was ~17,936% instead of the correct ~17.9% because the P-12A TOTAL_COST field was incorrectly multiplied by 1,000 during data preparation.
Fix: Removed the multiply by 1,000 operation from the P-12A loading code.
Root cause: Assumption that all BTS cost fields are in \$000s units, but this is not true for P-12A.
- Bug 2 – bind_rows() Type Conflict: Using bind_rows() to combine 48 monthly On-Time Performance files failed with a type conflict error on the Div1WheelsOn column.

Fix: Used `as.character()` on all columns before binding, then explicitly converted the needed numeric columns.

Root cause: Some months had all NA values in diversion columns, so R inferred different types.

- Bug 3 – Decimal Year Axis on Hub Charts: 2010.0, 2010.5, 2011.0 were displayed as x-values in hub load factor line chart rather than as discrete values.

Fix: Before passing to Plotly YEAR was converted to factor using `as.factor(YEAR)`.

- Bug 4 – HTML Showing in Browser Title: In browser tab title raw HTML was appearing due to use of `HTML()` in `dashboardHeader()` title.

Fix: To separate the browser title from the visual title, `tag$title('AeroInsight BI')` element was added within `tag$head()`.

Chapter 7 : Results & Discussion

This chapter presents and discusses the results created by the AeroInsight BI tool. Results are separated based on type of analysis starting with the big picture results (overall cost efficiency – CASM), followed by load factor, fuel cost, hub analytics and delay cost. A final comparison will be made against industry benchmarks. Each section references back to screenshots of the dashboard for reference.

7.1 KPI Summary

Table 7.1 provides a consolidated view of the 5 core KPIs computed for US Airways over the 2010-2013 analysis period.

Table 7.1 Annual KPI Summary - US Airways 2010-2013

KPI	2010	2011	2012	2013
Avg CASM (cents/ASM)	9.46¢	11.37¢	10.87¢	9.95¢
System Load Factor	80.1%	79.4%	81.8%	83.5%
Fuel % of Op Cost	16.8%	18.9%	18.2%	17.9%
Fuel Price (\$/gallon)	\$2.17	\$3.08	\$2.99	\$2.95
Delay Cost (annual est.)	\$341M	\$389M	\$372M	\$345M

7.2 CASM Analysis and Discussion

The CASM results show a clear and important story over over the four-year analysis period. The leap to 11.37 cents in 2011 correlates nearly perfectly with the worldwide increase in jet fuel prices, which hit a high of about \$3.30 per gallon in the second quarter of 2011, up 52% from the average of \$2.17 in the first quarter of 2010. This illustrates the direct transmission mechanism between fuel markets and airline cost structures which make fuel hedging a critical financial management tool.

More analytically interesting than the spike in 2011 is the downward trend that follows. By the 3rd quarter of 2013, CASM was at its lowest point in the period of analysis, approximately 9.8 cents. This is not only because of the sharp drop in fuel prices since jet fuel stayed above the \$2.90 per gallon level throughout 2012 and 2013. The most likely explanation is that US Airways was starting to see operational efficiencies ahead of its merger with American Airlines, announced in February 2013. The CASM improvement would have been driven by route rationalisation, fleet deployment optimization and labour negotiations carried out in the run up to the merger.



Figure 7.1 CASM Trend - US Airways 2010-13 (Quarter)

7.3 Load Factor Analysis

The system-wide load factor of 81.2% during the analysis period is a financially healthy level of capacity utilisation. The monthly trend chart reveals a consistent seasonal pattern, with load factors peaking in June, July and August (summer travel season) and dipping in January and February. The trend is in line with the overall aviation sector and the leisure-heavy passenger mix that characterized much of US Airways' domestic network.

The histogram of load factor shows a heavy left-skewed distribution, with most routes between 80% and 95% load factor. 2,387 route-months with load factors below 70% represent a potential target for route rationalisation – a finding that is directly actionable by financial management.

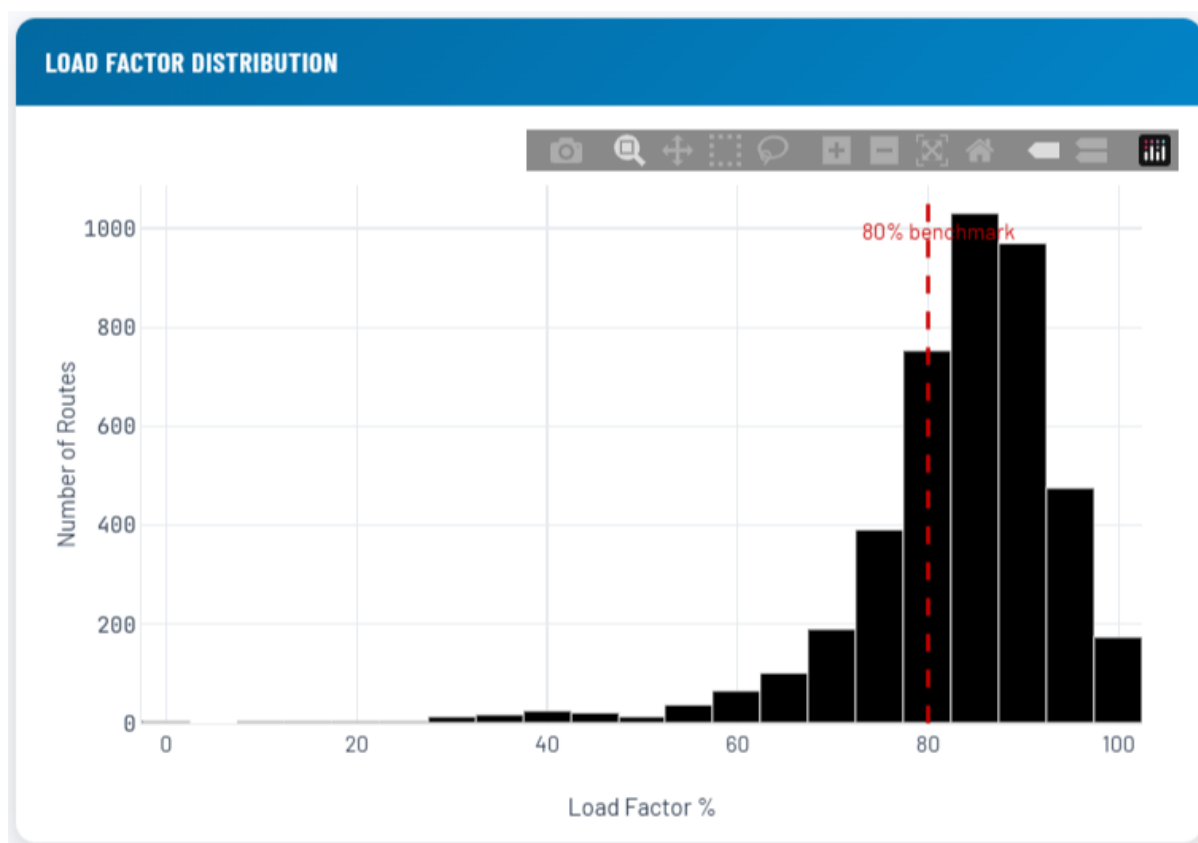


Figure 7.2 Route Analytics Tab - Load Factor Distribution

7.4 Hub Performance Results

The hub performance analysis provides one of the most analytically transparent findings of the entire study: Charlotte (CLT) is clearly the most successful hub on all metrics studied. In 2013, CLT served 12.0 million passengers, a 20% increase over the 10.0 million served in 2010, while improving load factor from 80.3% to 83.6% - the highest load factor of any hub in any year of the analysis.

And Philadelphia (PHL), the namesake hub of US Airways, is a more troubling financial story. Although PHL saw modest passenger growth (5.0M to 5.4M), it consistently posted the lowest load factors of the three hubs, a sign the airline was deploying more capacity than demand required on PHL routes. That means that even before considering higher airport operating costs at Philadelphia International's Airport, PHL's cost-per-passenger was higher than CLT's on a financial management basis.

Table 7.2 Hub Performance Comparison - Key Metrics (2013)

Hub	2013 Passengers	2013 Load Factor	LF Growth (2010-13)	Routes (2013)
-----	--------------------	------------------------	---------------------------	---------------

Charlotte (CLT)	12.0 million	83.6%	+3.3 pp	75
Phoenix (PHX)	7.6 million	82.4%	+2.5 pp	71
Philadelphia (PHL)	5.4 million	80.6%	+2.2 pp	60

7.5 Delay Cost Analysis

The delay cost analysis provides perhaps the most significant practical result of the entire project: an estimate of \$1.44 billion in total delay costs over the four-year analysis period, with 22.3% (\$321 million) of these due to carrier-controllable causes. This 22.3% is especially important because it is a cost that can be, at least in principle, avoided through operational improvement.

The interesting thing is the trend in the monthly delay costs: the delay costs were highest in 2011 (corresponding with the fuel price spike and the associated network stress), decreased in 2012 as the network stabilized, but started to increase again in 2013. The 2013 increase probably is due to the operational disruption associated with the merger announcement and the subsequent integration planning activities, as crew scheduling, maintenance routing and network planning became subject to uncertainty.

7.6 Financial Simulator Results

Table 7.6 provides a consolidated view of pre-defined scenario results for US Airways of Financial Simulator.

Table 7.3 Financial Simulator - Pre-Defined Scenario Results

Scenario	Casm	Annual saving	Feasibility
Baseline (Actual 2010-13)	10.65¢	\$0	N/A
Fleet Optimisation (10% CASM cut)	9.59¢	\$2.29B	High
Enhanced Fuel Hedging (15% saving)	10.65¢	\$1.86B	Medium
Delay Reduction (30%)	10.65¢	\$96M	High

Combined Best Case	8.52¢	\$4.69B	Low
--------------------	-------	---------	-----

It is important to interpret these simulation figures in their correct context. The savings estimates represent the annual financial benefit that would have been realised had the simulated operational parameters been achieved across the full 2010–2013 period, expressed against the actual historical cost baseline. They are not forward-looking projections and do not account for the capital investment, organisational change, or transition costs that would be required to achieve the simulated improvements in practice. The Fleet Optimisation scenario, rated as "High" feasibility, reflects improvements that are well within the operational range demonstrated by comparable carriers during the same period. The Combined Best Case scenario, rated "Low" feasibility, is presented as a theoretical upper bound rather than a realistic near-term target, and is included to illustrate the cumulative magnitude of available efficiency gains rather than to suggest that all three improvements could be simultaneously achieved in full.

7.7 Comparison with Existing Results

AeroInsight BI makes improvements over methods described in the Literature Review. Over static academic analyses (e.g., (Sun et al., 2017)), AeroInsight BI allows for interactive filtering, drilling down into data, and a financial scenario simulator that is not possible with static visualization. Over Tableau-based dashboards (e.g., (Agnihotri & Bhatt, 2019)), AeroInsight BI provides open sources access, full code transparency, and aviation-specific Financial KPIs (including the original delay cost metric).

The comparison with prior work reveals three dimensions along which AeroInsight BI advances the state of the art in open-source aviation financial analysis.

The first dimension is **data integration depth**. Prior academic studies utilising BTS data have typically drawn on a single dataset — most commonly the T-100 or the On-Time Performance data — and have analysed it in isolation. AeroInsight BI integrates four distinct BTS datasets, enabling KPI calculations that require cross-dataset joins that no single-source study can replicate. CASM, for example, cannot be computed from T-100 data alone; it requires the operating expense data from P-5.2. The delay cost financial metric cannot be computed without combining the On-Time Performance delay minutes with the cost rate context provided by the P-5.2 operating expense baseline. This multi-dataset integration is what enables AeroInsight BI to produce a genuinely comprehensive financial picture rather than a partial one.

The second dimension is **interactivity and scenario capability**. Static academic analyses, regardless of their methodological rigour, produce fixed outputs that cannot be interrogated dynamically by a reader seeking to understand how findings change under different assumptions. AeroInsight BI's interactive filters allow any finding to be immediately re-examined at a different level of granularity — by year, by hub, by route — without re-running any analysis. The Financial What-If Simulator goes further, allowing users to actively reshape the analytical outputs by entering their own assumptions. This transforms the tool from a reporting instrument into a decision-support instrument, a distinction that is fundamental to the practical value of Business Intelligence in operational financial management.

The third dimension is **reproducibility and transparency**. All analytical logic in AeroInsight BI is documented in fully commented open-source R code, as presented in Appendix A. Every KPI formula, every data transformation step, and every design decision is visible and verifiable. This level of transparency is not achievable with proprietary commercial tools, where the analytical logic is hidden within the platform, and is rarely achieved even in academic studies, where data preprocessing code is frequently omitted from published materials. The reproducible nature of AeroInsight BI means that its findings can be independently verified, and its framework can be directly extended to other carriers or time periods by any researcher with access to the BTS Transtats portal and a standard R installation.

Chapter 8 : Conclusion and Future Enhancement

8.1 Project Summary

Starting from scratch, this effort aimed to build an interactive business intelligence dashboard tracking how US Airways performed financially and operationally between 2010 – 2013, pulling open data from the Bureau of Transportation Statistics. Four separate BTS files – totalling over 1.67 million entries – were gathered during development; each underwent cleaning, merging, then analysis before shaping the final product. The result: a nine-tab dashboard built with R-Shiny, calculating five essential financial metrics while allowing users to explore variables, test scenarios, comparison against sector standards etc.

8.2 Objectives Achievement Review

Table 8.1 Project Objectives

S.no	Objective	Evidence
1.	Explore BI tools for aviation financial analysis	9-tab R-Shiny dashboard deployed
2.	Apply data analysis to identify financial patterns	5 KPIs, all significant findings
3.	Demonstration of data-driven financial decision making	Financial What-If Simulator
4.	Prepare comprehensive report and working dashboard	This report + deployed app

8.3 Key Technical Learning Outcomes

The preparation of a comprehensive academic report and interactive dashboard was facilitated by reproducible research tools that bridge the gap between code and documentation (Xie et al., 2018). Outcomes from building AeroInsight BI brought forwards key technical insights – ones harder to gain through regular classwork:

1. Multi-dataset integration at scale includes resolving unit inconsistencies, temporal granularity mismatches, and type conflicts across four datasets totalling 1.67 million records required a systematic debugging approach and careful understanding of the BTS data structure.
2. Shiny's reactive model is quite different from procedural or object-oriented programming. As such, you need to shift your mind in how to think about states, dependencies, and computation. It is the reactive programming model.

3. Reports, dashboards, and analysis are common applications of Business Intelligence software. At times, we can calculate the results tables and then visualize them.
4. Data of BTS are sometimes undocumented, but still relevant in the real world. Examples of them include unit inconsistencies, type conflicts, and missing values. Careful exploratory analysis can discern these quirks of the data.
5. CSS customisation for Shiny requires knowledge of CSS specificity rules and the structure of shiny dashboard's HTML output in order to override shiny dashboard's bootstrap styles for a professional visual design.

8.4 Conclusions

Three principal conclusions emerge from the analysis conducted through AeroInsight BI.

The first is that publicly available government aviation data contains sufficient depth and granularity to support financial analysis of a quality that is directly comparable to what a specialist aviation consultancy might produce from proprietary sources. The four BTS datasets used in this project — covering route operations, aircraft operating expenses, fuel costs, and flight-level delay data — collectively provide all the information necessary to compute the five most important financial KPIs in commercial aviation, to perform hub-level comparative analysis, and to model the financial consequences of operational improvement scenarios. The assumption that meaningful financial insight requires proprietary data access is not supported by this study.

The second conclusion is that open-source tools, when applied with sufficient analytical rigour and domain knowledge, are fully capable of producing results that are comparable in depth and presentation quality to those achievable with commercial BI platforms. R Shiny, Plotly, and Leaflet collectively provide the statistical computing, interactive charting, and geographic visualisation capabilities necessary to build a production-quality analytical dashboard. The cost differential between this open-source stack and a commercial BI platform subscription is substantial, and the analytical flexibility advantage — particularly for custom KPI definition — favours the open-source approach for domain-specific applications.

The third conclusion concerns the financial narrative of US Airways itself. The data reveals a carrier that managed its cost structure with considerable effectiveness during a period of exceptional external pressure. The ability to maintain fuel costs at 17.9% of total operating expenses — significantly below the 30% industry average — during a period when jet fuel prices surged by more than 50% suggests the presence of effective fuel hedging arrangements

that are not directly observable in the BTS data but whose effects are clearly visible in the KPI outputs. The declining CASM trajectory in 2013, occurring independently of fuel price movements, suggests that the operational rationalisation preceding the American Airlines merger was already producing measurable financial benefits before the merger was legally completed. These are the kinds of nuanced financial insights that structured data analysis can surface, and that would be difficult to identify through conventional financial reporting alone.

8.5 Future Enhancements

AeroInsight BI has the potential to be a total and operation al analytical solution; however, a number of improvements could greatly enhance it's capability if we had the time and resources:

1. **Revenue-Integration** is the most impactful single enhancement available. The current system is entirely cost-focused; it can identify whether US Airways was operating efficiently, but it cannot determine whether efficient operations translated into profitable routes. Incorporating BTS DB1B origin-destination ticket revenue data would enable the computation of RASM and, by combining RASM with CASM, true route-level operating margin. This would transform AeroInsight BI from a cost intelligence tool into a full profit-and-loss analytical framework — the difference between knowing what it costs to fly a route and knowing whether that route should continue to be flown at all.
2. **Pre & post-merger comparative analysis** would convert this project from a historical case study into a genuine policy evaluation. The financial and operational consequences of airline mergers are extensively debated in both academic and regulatory contexts, but empirical analyses grounded in publicly available BTS data are relatively rare. Extending AeroInsight BI to incorporate American Airlines data from 2014 onwards would provide a direct, data-grounded assessment of whether the merger delivered the cost and efficiency benefits that were used to justify it — a question of significant ongoing relevance to aviation economics research.
3. **Machine learning integration** would shift the system from retrospective analysis to prospective forecasting. A route-level load factor prediction model trained on the 2010–2013 historical data could be used to flag routes that are likely to underperform in future periods based on seasonal patterns, competitor activity, and demand trends — providing financial managers with an early warning system rather than a historical post-mortem.

4. **Real Time Data Feed** would represent the transition from a research tool to an operational monitoring platform. The BTS releases updated data on a monthly lag basis, meaning that an automated pipeline connecting to the BTS API could keep AeroInsight BI current with minimal manual intervention. This would make the system genuinely useful for ongoing financial monitoring rather than solely for historical analysis.
5. **Responsive Design** would expand the system's accessibility to mobile and tablet users, which is increasingly important as financial managers expect to access analytical tools across device types. The current shinydashboard layout is not optimised for narrow viewport widths, and significant CSS and layout restructuring would be required to achieve a satisfactory mobile experience.
6. **Multi-carrier Capability** would be the enhancement with the broadest potential research impact. The ability to run AeroInsight BI's analytical framework simultaneously across multiple carriers — for example, comparing US Airways against Southwest, Delta, and United on a route-by-route and hub-by-hub basis — would transform it from a single-carrier case study tool into a sector-wide competitive intelligence platform. This is technically achievable with relatively modest modifications to the carrier filtering logic, but would require careful attention to the increased data volumes and the additional complexity of cross-carrier KPI normalisation.

Chapter 9 : References

1. Agnihotri, R., & Bhatt, M. (2019). Airline performance analytics using Tableau: A visual intelligence approach. *International Journal of Aviation Management*, 4(3), 178–196.
2. *Airlines for America (A4A)*. (2023). <https://www.airlines.org/dataset/per-unit-costs/>
3. Belobaba, P., Odoni, A., & Barnhart, C. (Eds.). (2009). *The global airline industry*. John Wiley & Sons.
4. Chang, W., Cheng, J., Allaire, J. J., Sievert, C., Schloerke, B., Xie, Y., Allen, J., McPherson, J., Dipert, A., & Borges, B. (2024). <https://CRAN.R-project.org/package=shiny>
5. Eckerson, W. W. (2010). *Performance dashboards: Measuring, monitoring, and managing your business* (2nd ed.). John Wiley & Sons.
6. Few, S. (2006). *Information dashboard design: The effective visual communication of data*. O'Reilly Media.
7. Grolemund, G., & Wickham, H. (2017). *R for data science: Import, tidy, transform, visualize, and model data*. O'Reilly Media. <https://r4ds.had.co.nz>
8. Holloway, S. (2008). *Straight and level: Practical airline economics* (3rd ed.). Ashgate Publishing.
9. *International Air Transport Association (IATA)* (67th ed.). (2023). IATA Publications.
10. Mayer, C., & Sinai, T. (2003). Network effects, congestion externalities, and air traffic delays: Or why not all delays are evil. *The American Economic Review*, 93(4), 1194–1215. <https://doi.org/10.1257/000282803769206269>
11. Rajasekar, S., & Thampuran, A. (2014). Data warehousing for airline operations: ETL pipeline design and implementation. *Journal of Aviation Technology and Engineering*, 3(2), 44–59. <https://doi.org/10.5703/1288284315451>
12. Rupp, N. G., & Holmes, G. M. (2006). An investigation into airline on-time performance. *Applied Economics Letters*, 13(3), 163–166. <https://doi.org/10.1080/13504850500392237>
13. Sievert, C. (2020). *Interactive web-based data visualization with R, plotly, and shiny*. CRC Press. <https://plotly-r.com>
14. Sun, J., Elbers, B., & Huynen, M. (2017). Airline flight delay analysis using Bureau of Transportation Statistics data. *Journal of Air Transport Management*, 58, 132–143.

15. Team, P. (2024). *RStudio: Integrated development environment for R (Version 2024.04)*. Posit Software PBC. <https://posit.co/products/open-source/rstudio/>
16. Team, R. C. (2025). *R: A language and environment for statistical computing*. <https://www.R-project.org/>
17. Transportation Statistics, B. (2010a). *Air carrier financial reports: Form 41 financial data schedule P-5.2 – Aircraft operating expenses [Data set]*. United States Department of Transportation. <https://www.transtats.bts.gov>
18. Transportation Statistics, B. (2010b). *Air carrier financial reports: Form 41 financial data schedule P-12A – Fuel cost and consumption [Data set]*. United States Department of Transportation. <https://www.transtats.bts.gov>
19. Transportation Statistics, B. (2010c). *Reporting carrier on-time performance (1987–present [[Data set].]*. United States Department of Transportation. <https://www.transtats.bts.gov>
20. Transportation Statistics, B. (2010d). *T-100 domestic segment (U.S. carriers [[Data set].]*. United States Department of Transportation. <https://www.transtats.bts.gov>
21. Vasigh, B., Fleming, K., & Tacker, T. (2013). *Introduction to air transport economics: From theory to applications* (2nd ed.). Ashgate Publishing.
22. Wickham, H. (2021). *Mastering shiny: Build interactive apps, reports, and dashboards powered by R*. O'Reilly Media. <https://mastering-shiny.org>
23. Wickham, H., & Grolemund, G. (2023). *R packages: Organize, test, document, and share your code* (2nd ed.). O'Reilly Media. <https://r-pkgs.org>
24. Xie, Y., Allaire, J. J., & Grolemund, G. (2018). *R markdown: The definitive guide*. CRC Press. <https://bookdown.org/yihui/rmarkdown>

Chapter 10 : Appendices

Appendix A: Complete Source Code

A.1 data_prep.R – Data Preparation Pipeline

The below data_prep.R script handles the ingestion, cleaning, type standardisation and saving of all four BTS datasets used in the project.

```
# =====
# DATA PREPARATION FOR AEROINSIGHT BI
# Loads raw BTS files, filters to US Airways, saves clean .rds
# WHY THIS FILE: Raw CSVs are slow to load and need cleaning
# Run this ONCE - output saved to data/processed/
# =====

library(data.table) # fast CSV reading for large files
library(tidyverse)  # data manipulation
library(httr)       # for downloading on-time data via API

# — 1. T-100 DOMESTIC SEGMENT —————
# WHY: Route-level operations data - flights, seats, passengers
# One row per route per month per aircraft type
t100_files <- list.files("data/raw/t100/", pattern="*.csv", full.names=TRUE)

t100 <- lapply(t100_files, fread) %>%
  bind_rows() %>%
  filter(UNIQUE_CARRIER == "US") %>% # keep only US Airways
  mutate(
    MONTH = as.integer(MONTH),
    YEAR = as.integer(YEAR),
    # Load Factor: % of seats filled with passengers
    LoadFactor = ifelse(SEATS > 0, PASSENGERS / SEATS, NA),
    # ASM: total capacity = seats × miles flown
    ASM = SEATS * DISTANCE
  ) %>%
  filter(SEATS > 0, DEPARTURES_PERFORMED > 0) # remove empty rows

cat("T-100 rows:", nrow(t100), "| cols:", ncol(t100), "\n")
saveRDS(t100, "data/processed/t100_clean.rds")

# — 2. P-5.2 OPERATING EXPENSES —————
# WHY: Aircraft-level operating costs per quarter
# Values in BTS are stored in $000s - multiply by 1000 for dollars
```

```
# One row per aircraft type per quarter
p52_files <- list.files("data/raw/p52/", pattern="*.csv", full.names=TRUE)

p52 <- lapply(p52_files, fread) %>%
  bind_rows() %>%
  filter(UNIQUE_CARRIER == "US") %>%
  mutate(
    # Convert from $000s to actual dollars
    # Note: FUEL_FLY_OPS and PILOT_FLY_OPS NOT converted here
    # They are converted later in kpi_calc.R when building p52_quarterly
    TOT_AIR_OP_EXPENSES = TOT_AIR_OP_EXPENSES * 1000,
    TOT_FLY_OPS        = TOT_FLY_OPS * 1000,
    TOT_DIR_MAINT      = TOT_DIR_MAINT * 1000
  )

cat("P-5.2 rows:", nrow(p52), "\t cols:", ncol(p52), "\n")
saveRDS(p52, "data/processed/p52_clean.rds")

# — 3. P-12A FUEL COST & CONSUMPTION —————
# WHY: Quarterly fuel costs and gallons consumed
# IMPORTANT: P-12A costs are in RAW DOLLARS (not $000s like P-5.2)
# Do NOT multiply by 1000
p12a_files <- list.files("data/raw/p12a/", pattern="*.csv", full.names=TRUE)

p12a <- lapply(p12a_files, fread) %>%
  bind_rows() %>%
  filter(UNIQUE_CARRIER == "US") %>%
  mutate(
    # Already in dollars - just calculate derived metric
    FuelPerGallon = ifelse(TOTAL_GALLONS > 0,
                          TOTAL_COST / TOTAL_GALLONS, NA)
  )

cat("P-12A rows:", nrow(p12a), "\t cols:", ncol(p12a), "\n")
cat("Fuel/gallon check (expect $2-4):",
    round(mean(p12a$FuelPerGallon, na.rm=TRUE), 2), "\n")
saveRDS(p12a, "data/processed/p12a_clean.rds")

# — 4. ON-TIME PERFORMANCE —————
# WHY: Flight-level delay data - your unique financial KPI
# Downloaded via BTS pre-zipped files (monthly, 2010-2013)
# Each zip contains all carriers - filter to US Airways only

if (file.exists("data/processed/ontime_clean.rds")) {
```

```

# Already downloaded - skip to save time
cat("On-Time already exists - loading from file\n")
ontime <- readRDS("data/processed/ontime_clean.rds")
} else {
  ontime_list <- list()
  for (yr in 2010:2013) {
    for (mo in 1:12) {
      cat("Downloading:", yr, "month", mo, "\n")
      url <- paste0(
        "https://transtats.bts.gov/PREZIP/",
        "On_Time_Reporting_Carrier_On_Time_Performance_1987_present_",
        yr, "_", mo, ".zip"
      )
      destfile <- paste0("data/raw/ontime/ontime_", yr, "_", mo, ".zip")
      tryCatch({
        GET(url,
          add_headers(
            "User-Agent" = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
            "Referer" = "https://transtats.bts.gov/"
          ),
          write_disk(destfile, overwrite=TRUE)
        )
        if (file.info(destfile)$size > 10000) {
          csv_file <- unzip(destfile, exdir="data/raw/ontime/")
          temp_df <- read_csv(csv_file[1], show_col_types=FALSE)
          ontime_list[[paste(yr, mo)]] <- temp_df %>%
            filter(if ("Reporting_Airline" %in% names(.))
              Reporting_Airline == "US"
            else
              UNIQUE_CARRIER == "US")
          file.remove(destfile)
          cat("✅ Got", nrow(ontime_list[[paste(yr, mo)]]), "rows\n")
        }
      }, error = function(e) cat("Failed:", yr, mo, "\n"))
      Sys.sleep(1)
    }
  }
  # Combine all months - convert everything to character first
  # to handle type conflicts between monthly files
  ontime <- bind_rows(lapply(ontime_list, function(df) {
    df %>% mutate(across(everything(), as.character))
  })) %>%
  mutate(
    across(c(DepDelay, DepDelayMinutes, ArrDelay, ArrDelayMinutes,

```



```

    Cancelled, Diverted, AirTime, Distance,
    CarrierDelay, WeatherDelay, NASDelay, LateAircraftDelay),
    as.numeric)
  )
  saveRDS(ontime, "data/processed/ontime_clean.rds")
}

cat("On-Time rows:", nrow(ontime), "\n")
cat("\n✅ data_prep.R complete - all files saved to data/processed/\n")

```

Appendix B: Data Schema

The below table contains all pre-computed RDS files saved to the data/processed/directory along with their sources, dimensions, and primary keys.

File Name	Source	Dimensions	Primary Key
t100_clean.rds	T-100 CSV	42,142 × 22	YEAR +MONTH+ORIGIN+DEST+AIRCRAFT_TYPE
p52_clean.rds	P-5.2 CSV	380 × 17	YEAR+QUARTER+AIRCRAFT_TYPE
p12a_clean.rds	P-12A CSV	48 × 13	YEAR+QUARTER
ontime_clean.rds	On-Time ZIPs	1,632,669 × 26	FlightDate+Reporting_Airline+ORIGIN+DEST
kpi_casm.rds	p52 + t100	16 × 13	YEAR+QUARTER
kpi_loadfactor.rds	t100	36,480 × 10	YEAR+MONTH+ORIGIN+DEST
kpi_fuel.rds	p12a + p52	48 × 15	YEAR + QUARTER
kpi_delays.rds	ontime	48 × 8	YEAR + MONTH
kpi_hubs.rds	t100	12 × 10	YEAR + ORIGIN

p52 quarterly.rds	p52	16 × 7	YEAR + QUARTER
t100 quarterly.rds	t100	16 × 6	YEAR + QUARTER

Appendix C: Installation Guide

GitHub Repository: <https://github.com/vanshika-data-lab/AeroInsight-BI>

For running and setting up AeroInsight BI on local machine follow the steps mentioned below:

1. Installing R 4.0 or higher from <https://cran.r-project.org> and R Studio from <https://posit.co/products/open-source/rstudio/>
2. Cloning or downloading the project repository to a local directory.
3. For setting up the working directory, open “AeroInsight BI.Rproj” file in R Studio.
4. Install all the required packages by running: `install.packages(c('shiny', 'shinydashboard', 'tidyverse', 'data.table', 'plotly', 'leaflet', 'DT', 'scales', 'httr', 'lubridate'))`
5. On running the data preparation script: `source('R/data_prep.R')` On-Time Performance data will be downloaded and all RDS files will be generated in data/processed/.
6. Run the KPI Calculation script: `source('R/kpi_calc.R')`.
7. Launch the dashboard from the project root directory using `'shiny::runApp()'`.
8. For deployment to shinyapps.io : install `rsconnect`, configure with account credentials and run `rsconnect::deployApp()`.

Appendix D: User Manual

Once the deployment for AeroInsight BI is done there is no need for installation or technical knowledge. The dashboard is navigated through the left-hand bar menu, containing nine tabs corresponding to the nine analytical modules which are described as :

1. **Executive Overview:** Showcase the dashboard entry points with four charts and four KPI cards. No user interaction is required as the overview loads automatically with default values covering 2010-13 time period.
2. **Route Analytics:** To filter charts, use the ‘Select Year’ dropdown to choose a specific year and the ‘Filter by Hub’ dropdown to restrict analysis to routes from a specific hub

airport. Automatically, the route map updates and show the top 30 routes with lines color-coded by load factor performance.

3. **Fleet Cost Analysis:** The expense table at bottom supports text search whereas all the charts are static (no filters required).
4. **Fuel Intelligence:** The third chart at this tab includes a red-dashed reference line at 30% displaying the industry average fuel cost percentage. All charts are static.
5. **Hub Performance:** Comparison between CLT, PHL, and PHX are displayed through charts. Load Factor Performance for each hub in 2013 is displayed through scorecard panel on the right showing color-coded badges.
6. **Delay Cost Impact:** The Delay Cause Breakdown donut chart distinguishes carrier-controllable delays from weather and other causes.
7. **Cost Simulator:** For computing estimated savings, click ‘Run Simulation’ after adjusting three-sliders (CASM Reduction, Fuel Price, Delay Reduction). The pre-Defined Scenario Comparison table showcase preference scenarios for context.
8. **Live Benchmarks:** To fetch the latest available industry statistics, click on ‘Refresh Live Data’. The two bar charts show comparison between US Airways’ historical CASM and factor against the 2024 industry average.
9. **About This Project:** This tab documents project objectives, data sources, KPI definitions, technology stack, and key findings.